

Supercharge Your C++ Project

10 TIPS TO ELEVATE FROM REPO TO PROFESSIONAL PRODUCT

Mateusz Pusz

March 17, 2026



Question from a workshop participant

"Which project should I contribute to learn Modern C++?"

Question from a workshop participant

"Which project should I contribute to learn Modern C++?"

It depends...

The Current State of C++ Libraries

We have "Bleeding Edge" templates, **constexpr**, and concepts, but we have "Leading Edge" frustration.

The Current State of C++ Libraries

We have "Bleeding Edge" templates, **constexpr**, and concepts, but we have "Leading Edge" frustration.

Technical excellence $\neq$ Project excellence

The "Fortress" Library

THE SYMPTOMS



The "Fortress" Library

THE SYMPTOMS

- Hard to enter
 - "First, install Clang 18, CMake 3.28..."



The "Fortress" Library

THE SYMPTOMS

- **Hard to enter**
 - "First, install Clang 18, CMake 3.28..."
- **Hard to understand**
 - "Read the source, Luke."



The "Fortress" Library

THE SYMPTOMS

- **Hard to enter**
 - "First, install Clang 18, CMake 3.28..."
- **Hard to understand**
 - "Read the source, Luke."
- **Defensive maintainers**
 - "This is the correct design. Period."
 - "If you don't understand why, you shouldn't be contributing."



The "Fortress" Library

THE SYMPTOMS

- **Hard to enter**
 - "First, install Clang 18, CMake 3.28..."
- **Hard to understand**
 - "Read the source, Luke."
- **Defensive maintainers**
 - "This is the correct design. Period."
 - "If you don't understand why, you shouldn't be contributing."
- **No clear path for contributors**
 - "Just look at the code to see how it works."
 - No **CONTRIBUTING.md**, no first issues, ...

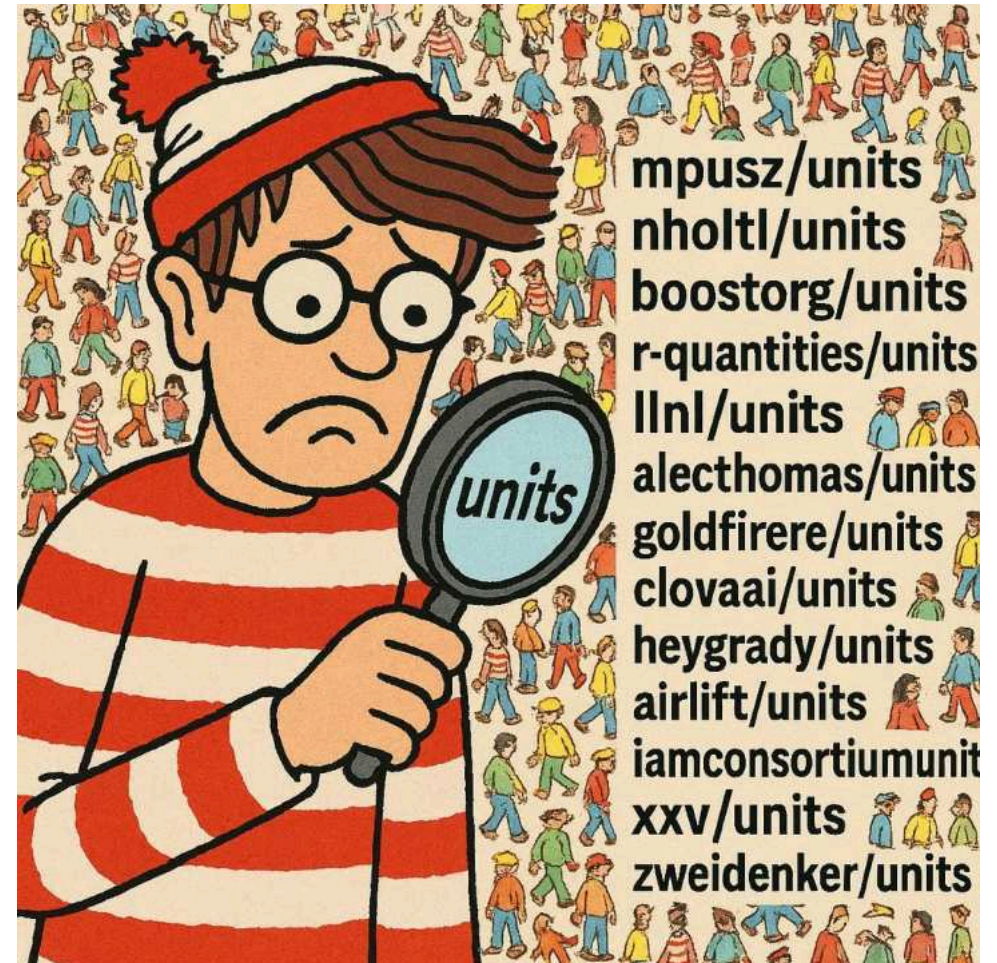


TIP 1

DISCOVERABILITY: DON'T BE A HIDDEN GEM

The Generic Name Problem

- GitHub is full of **units**, **http**, **json** libraries
- My old library: **mpusz/units** competed with **nholthaus/units**

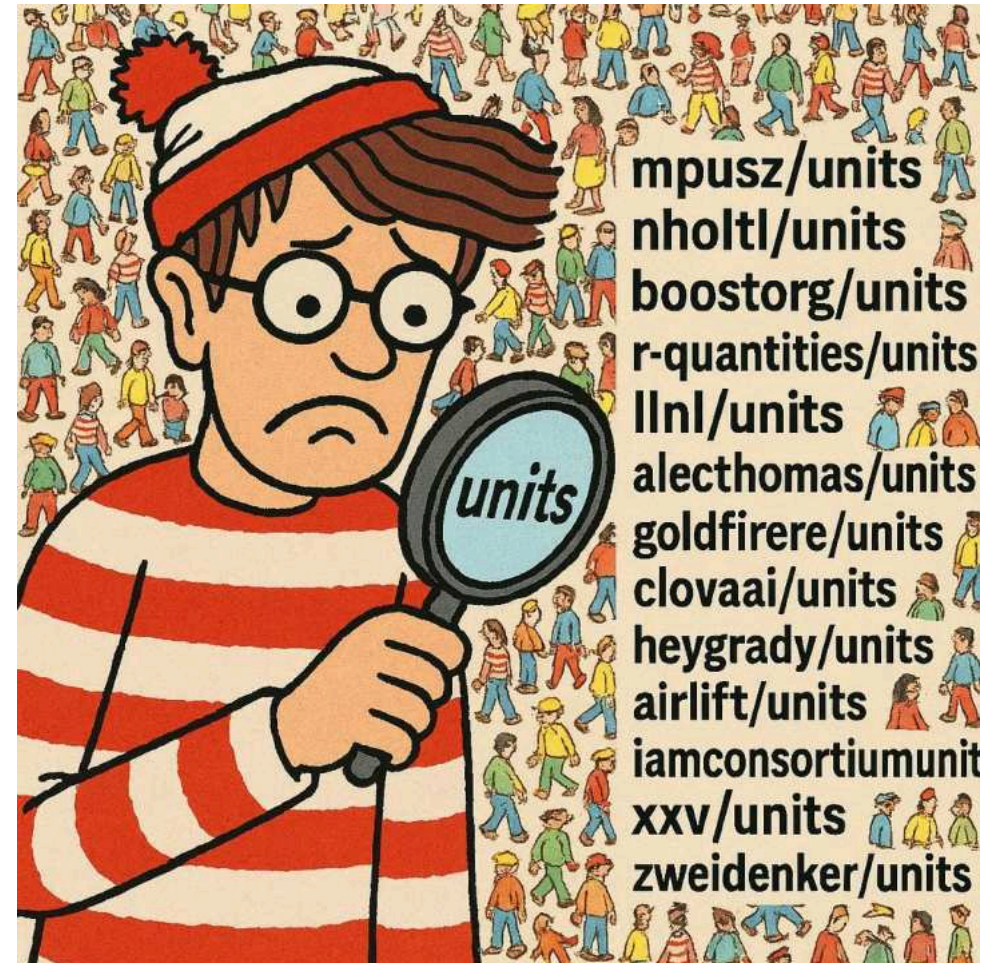


The Generic Name Problem

- GitHub is full of `units`, `http`, `json` libraries
- My old library: `mpusz/units` competed with `nholthaus/units`

PROBLEM

- Which one should users choose?
- Hard to search, hard to discuss



The Generic Name Problem

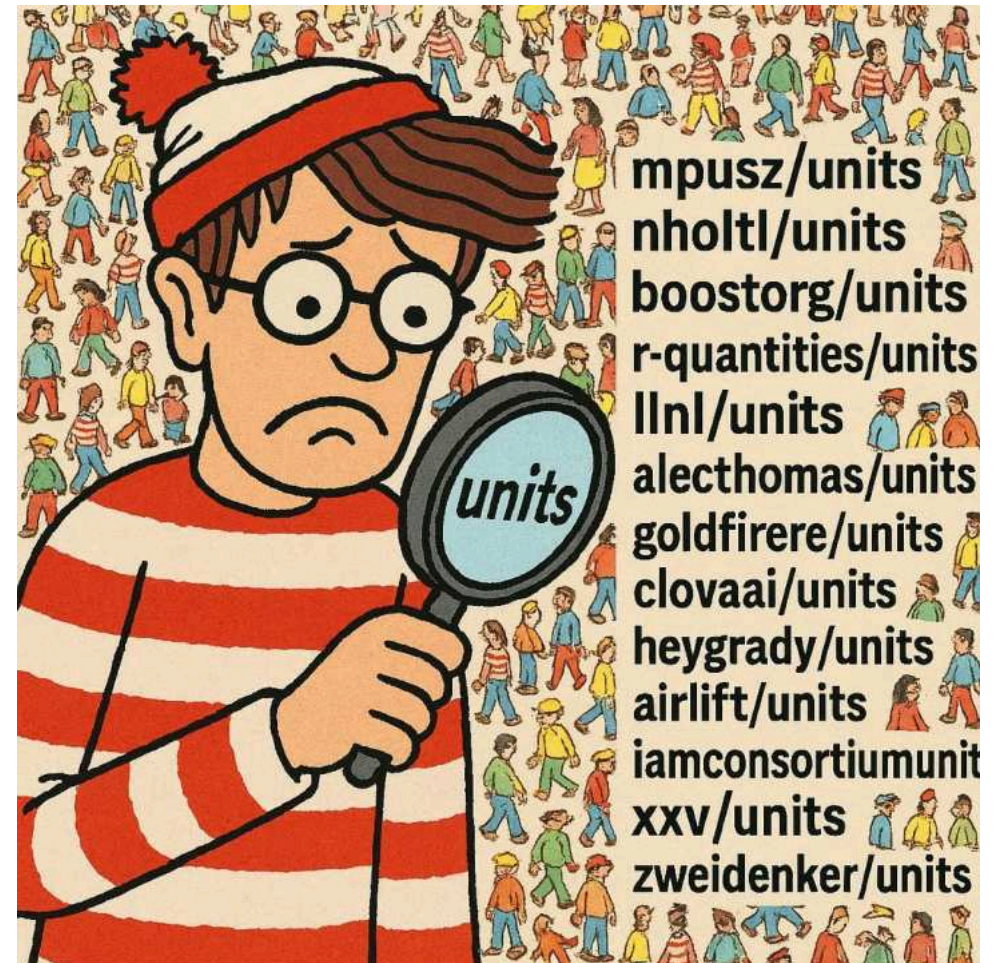
- GitHub is full of `units`, `http`, `json` libraries
- My old library: `mpusz/units` competed with `nholthaus/units`

PROBLEM

- Which one should users choose?
- Hard to search, hard to discuss

LESSON

- Generic names create discovery and branding problems



Naming is a Technical Decision

Avoid generic names like `units`, `linalg`, `http`.

Naming is a Technical Decision

Avoid generic names like `units`, `linalg`, `http`.

THE MP-UNITS BLUEPRINT

- **GitHub Repo:** `mpusz/mp-units`
- **CMake Target:** `mp-units::mp-units`
- **Conan Package:** `mp-units`
- **Namespace:** `mp_units`
- **Include path:** `#include <mp-units/systems/si.h>;`

Naming is a Technical Decision

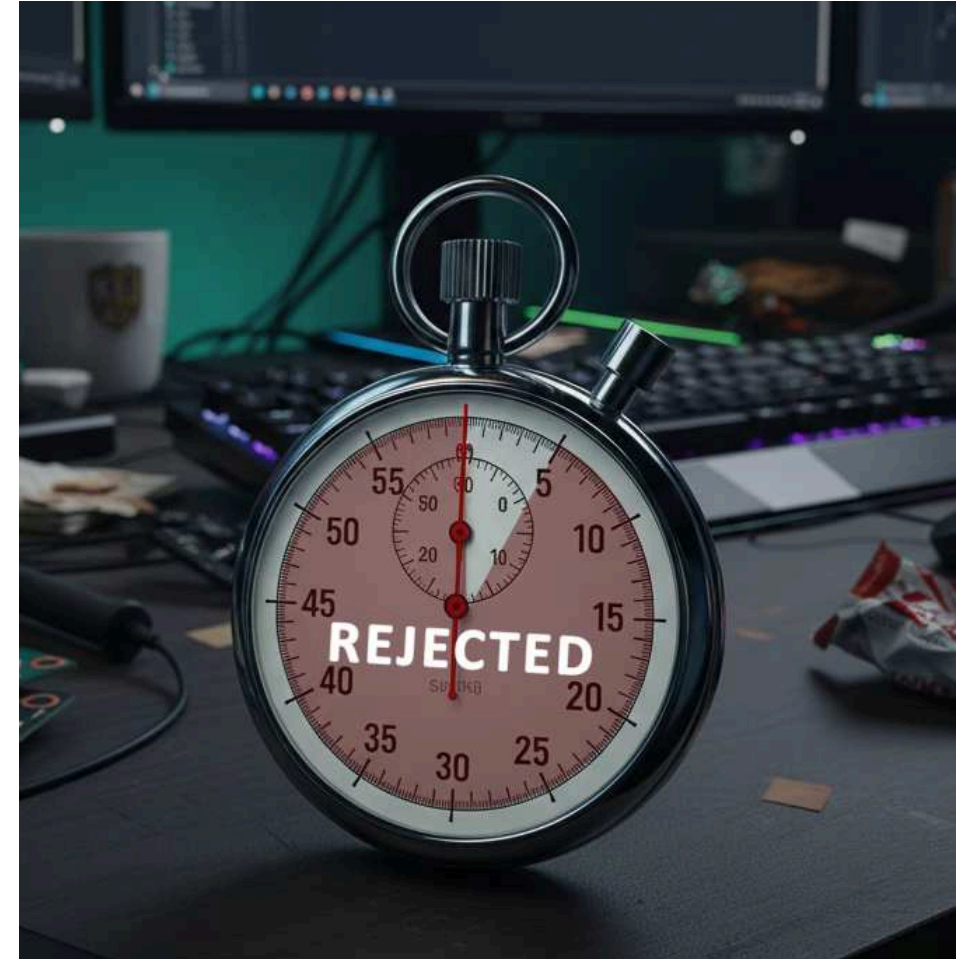
Avoid generic names like `units`, `linalg`, `http`.

THE MP-UNITS BLUEPRINT

- **GitHub Repo:** `mpusz/mp-units`
- **CMake Target:** `mp-units::mp-units`
- **Conan Package:** `mp-units`
- **Namespace:** `mp_units`
- **Include path:** `#include <mp-units/systems/si.h>;`

No ambiguity, easy to search, consistent branding.

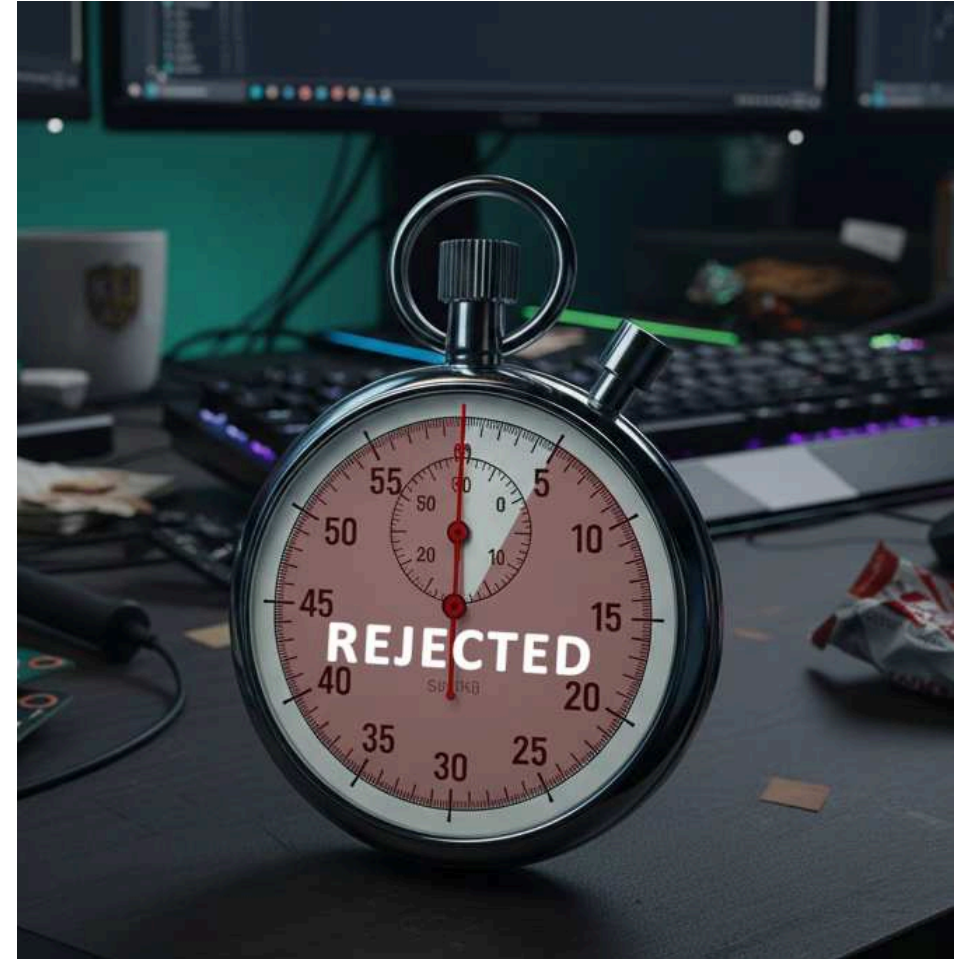
The 5-Second README Test



The 5-Second README Test

1 Logo & Badges

- Build status, coverage, license, version



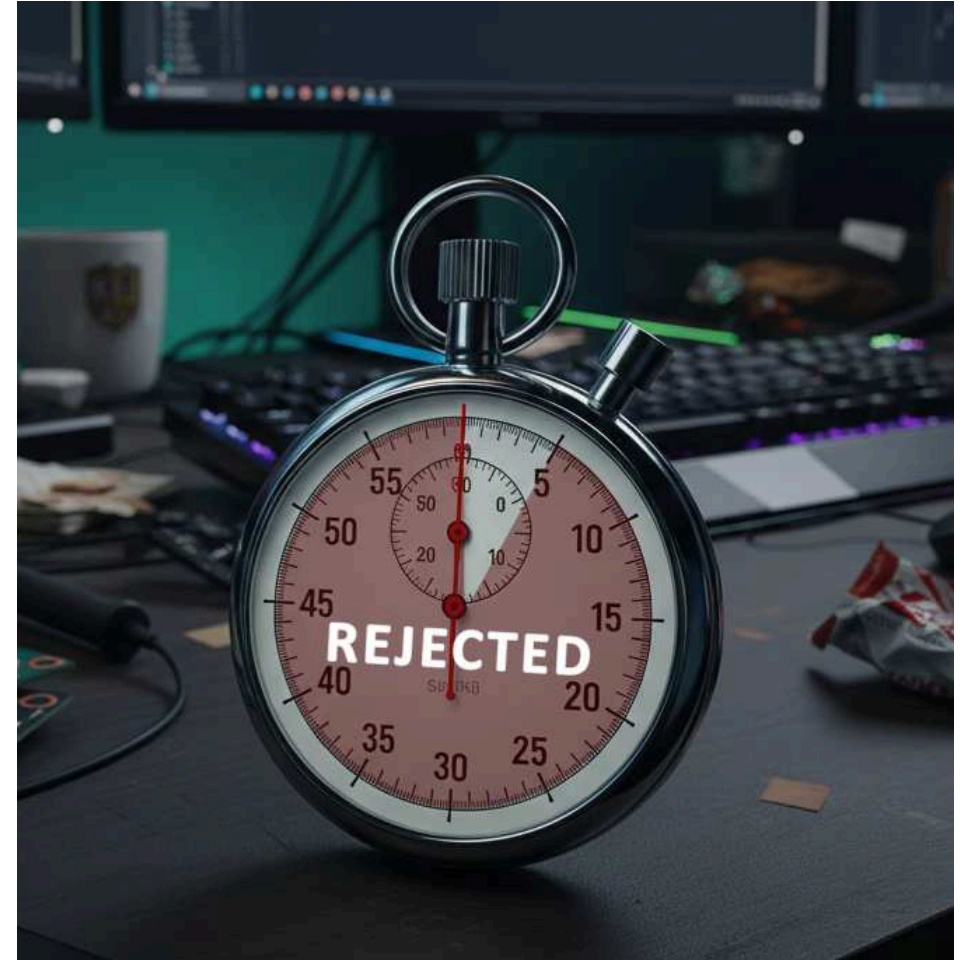
The 5-Second README Test

1 Logo & Badges

- Build status, coverage, license, version

2 The Pitch

- What problem do you solve?
- Not just "A units library"



The 5-Second README Test

1 Logo & Badges

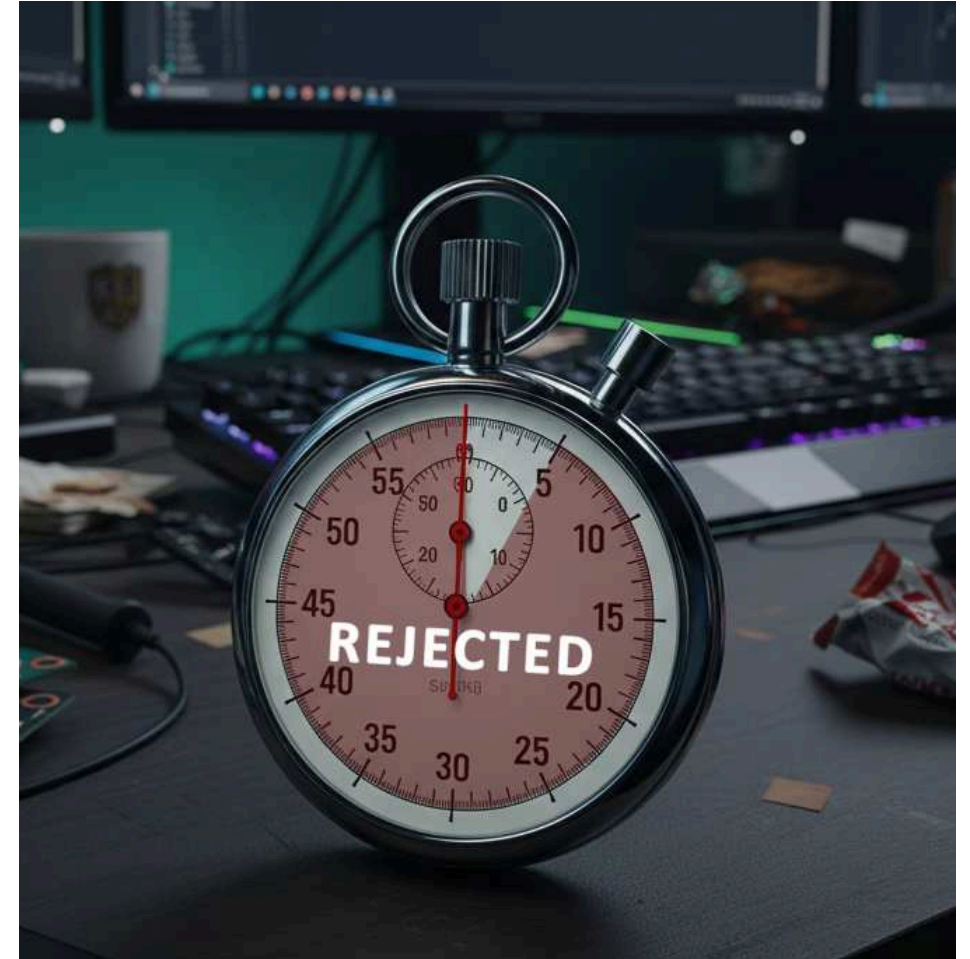
- Build status, coverage, license, version

2 The Pitch

- What problem do you solve?
- Not just "A units library"

3 Quick Start

- Inline code example visible without scrolling



The 5-Second README Test

1 Logo & Badges

- Build status, coverage, license, version

2 The Pitch

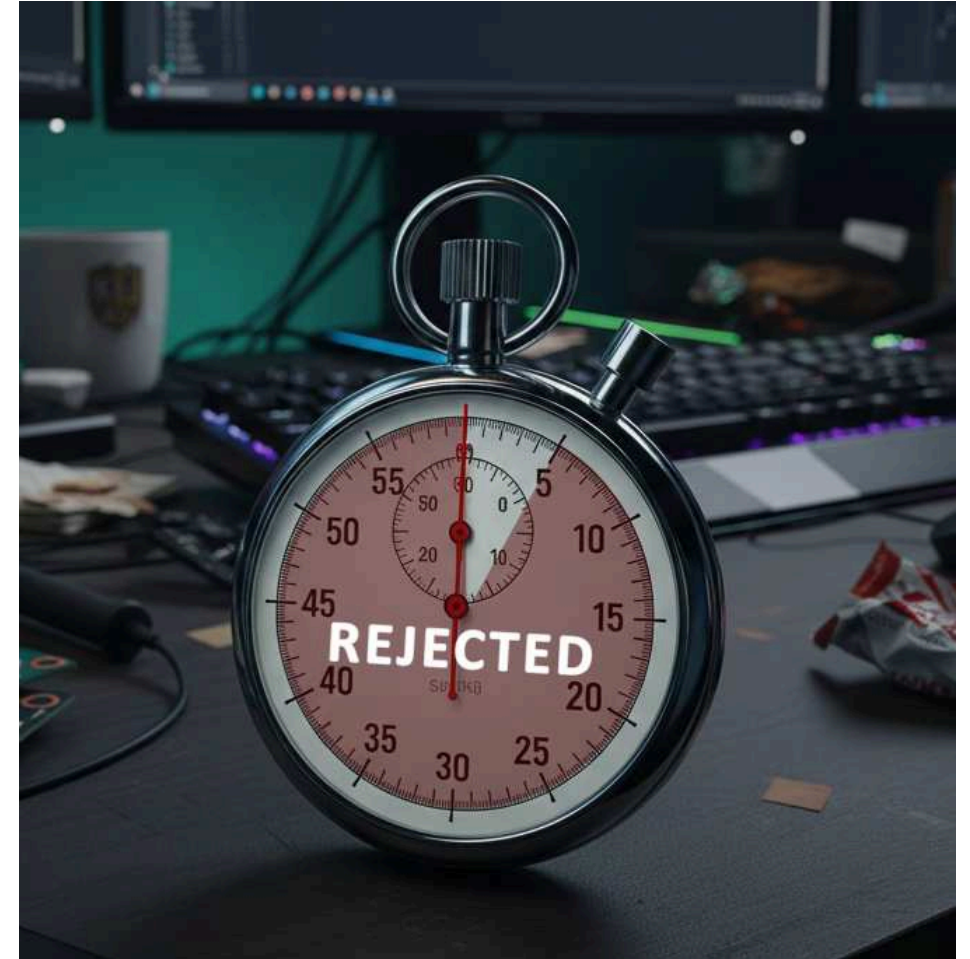
- What problem do you solve?
- Not just "A units library"

3 Quick Start

- Inline code example visible without scrolling

4 Live Demo

- Compiler Explorer / interactive link



Bad vs. Good Pitch

 BAD

"A units library for C++."

Bad vs. Good Pitch

 BAD

"A units library for C++."

 GOOD

"**mp-units** is the only Modern C++ (C++20 and later) library providing the full spectrum of compile-time safety for domain-specific quantities and units — from dimensional analysis to quantity kind safety — built on the ISO 80000 International System of Quantities (ISQ). It is the leading candidate for C++29 standardization — your chance to shape the future of C++."

Bad vs. Good Pitch

 BAD

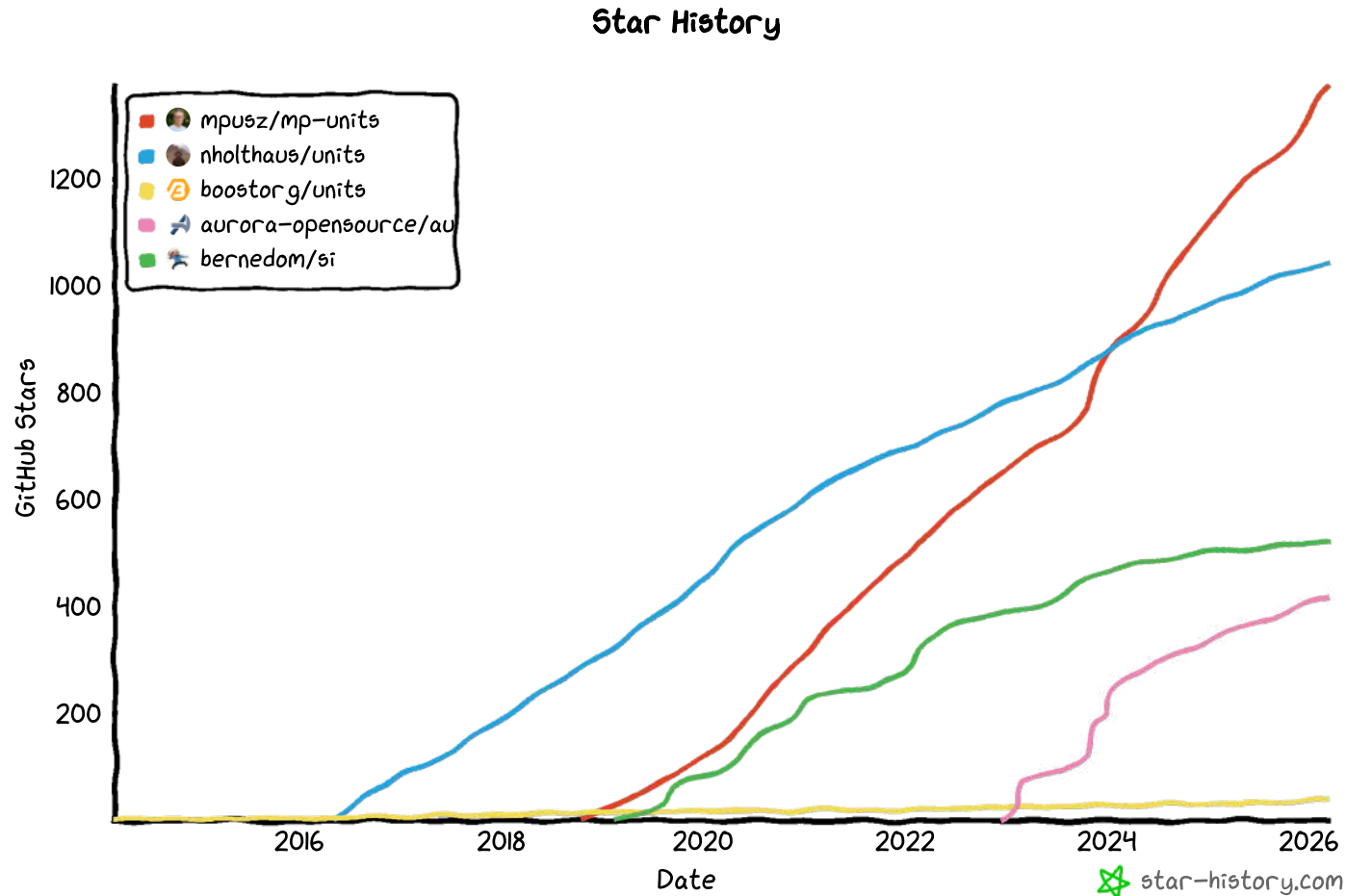
"A units library for C++."

 GOOD

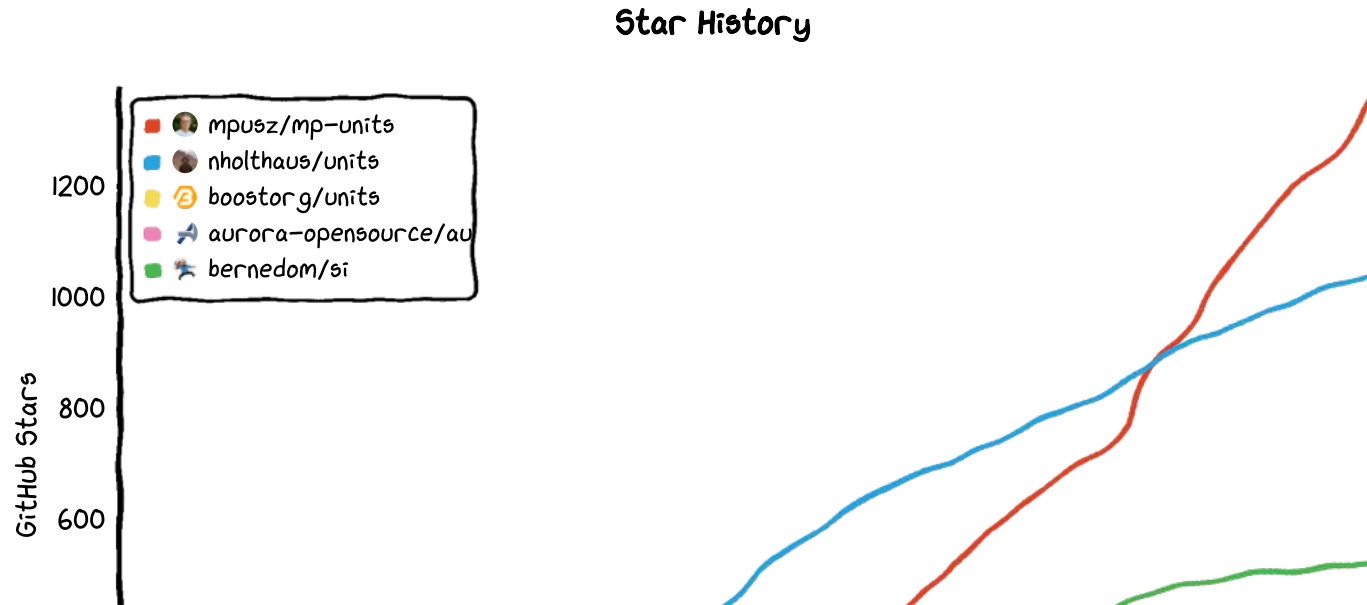
"**mp-units** is the only Modern C++ (C++20 and later) library providing the full spectrum of compile-time safety for domain-specific quantities and units — from dimensional analysis to quantity kind safety — built on the ISO 80000 International System of Quantities (ISQ). It is the leading candidate for C++29 standardization — your chance to shape the future of C++."

Key Difference: Specific, verifiable, differentiating claims.

Tracking Your Reach



Tracking Your Reach



Stars aren't just "vanity metrics" — they are a proxy for community trust and discoverability. Inflection points reveal what worked — use it to sharpen your strategy.

The Discoverability Blueprint

Don't be a hidden gem! If you don't advertise, it doesn't exist.

The Discoverability Blueprint

Don't be a hidden gem! If you don't advertise, it doesn't exist.

CHECKLIST

-  Unique, **searchable** project name
-  **Consistent naming** (namespace = CMake = package)
-  README with badges showing **"lights are on"**
-  **Code example** in first screen
-  Clear **value proposition**
-  **Track** your reach: **Star History**

TIP 2

THE "GREEN MASTER" PHILOSOPHY

The "Green Master" Philosophy

Trust is fragile! One broken build kills it.

The "Green Master" Philosophy

Trust is fragile! One broken build kills it.

Portability is not a "future goal"; it's a daily requirement.

The CI/CD Pipeline

GitHub Actions is the gold standard for Open Source.

The CI/CD Pipeline

GitHub Actions is the gold standard for Open Source.

COMPILERS

- GCC 12-15
- Clang 16–21
- Apple Clang 16
- MSVC 14.4

ARCHITECTURES

- x86-64, arm64

The CI/CD Pipeline

GitHub Actions is the gold standard for Open Source.

COMPILERS

- GCC 12-15
- Clang 16–21
- Apple Clang 16
- MSVC 14.4

ARCHITECTURES

- x86-64, arm64

CONFIGURATIONS

- **C++ std:** 20/23
- **Formatting:** fmtlib vs **std::format**
- **Contracts:** gsl-lite, ms-gsl, none
- **Build:** Debug/Release
- **Plus:** modules, import std, freestanding

Unit testing pipeline



Unit testing pipeline



The Problem: Testing all combinations = hundreds of jobs, timeouts, quota exhaustion. **The Solution:** Fuzz Your Build Matrix.

Fuzz Your Build Matrix

Generate build matrix for Conan CI summary

 **Build Matrix**

Random seed: 1125297464
Total jobs: 29

Toolchain	C++	Formatting	Contracts	Build	Modules	import_std	no_crt	freestanding
Apple Clang 16.1	20	std-format	gsl-lite	Debug	—	—	—	—
Apple Clang 16.1	23	std-format	gsl-lite	Debug	—	—	—	—
Clang-16 (x86-64)	20	fmtlib	gsl-lite	Debug	—	—	—	—
Clang-16 (x86-64)	20	fmtlib	ms-gsl	Release	—	—	—	—
Clang-17 (x86-64)	20	std-format	gsl-lite	Debug	—	—	—	—
Clang-17 (x86-64)	20	std-format	gsl-lite	Debug	■	—	—	—
Clang-18 (arm64)	20	std-format	gsl-lite	Debug	—	—	—	—
Clang-18 (arm64)	20	std-format	gsl-lite	Debug	■	—	—	—
Clang-18 (arm64)	23	fmtlib	gsl-lite	Release	■	—	■	—
Clang-18 (x86-64)	20	fmtlib	ms-gsl	Release	■	—	—	—
Clang-18 (x86-64)	20	std-format	gsl-lite	Debug	—	—	—	—
Clang-18 (x86-64)	20	std-format	gsl-lite	Debug	■	—	—	—
Clang-18 (x86-64)	23	std-format	none	Release	■	■	■	—
Clang-20 (x86-64)	20	std-format	gsl-lite	Debug	—	—	—	—
Clang-20 (x86-64)	20	std-format	gsl-lite	Debug	■	—	—	—
Clang-20 (x86-64)	23	std-format	none	Debug	■	■	■	—
Clang-21 (x86-64)	20	std-format	gsl-lite	Debug	—	—	—	—
Clang-21 (x86-64)	20	std-format	gsl-lite	Debug	■	—	—	—
Clang-21 (x86-64)	23	std-format	none	Debug	■	■	■	—
GCC-12	20	fmtlib	gsl-lite	Debug	—	—	—	—
GCC-12	20	fmtlib	ms-gsl	Release	—	—	—	—
GCC-13	20	std-format	gsl-lite	Debug	—	—	—	—
GCC-13	23	std-format	gsl-lite	Release	—	—	—	—
GCC-14	20	std-format	gsl-lite	Debug	—	—	—	—
GCC-14	23	fmtlib	none	Debug	—	—	■	—
GCC-15	20	std-format	gsl-lite	Debug	—	—	—	—
GCC-15	23	fmtlib	gsl-lite	Debug	—	—	■	—
MSVC 14.4	20	std-format	gsl-lite	Debug	—	—	—	—
MSVC 14.4	23	std-format	none	Release	—	—	■	—

Fuzz Your Build Matrix

Generate build matrix for Conan CI summary

Build Matrix

Random seed: 1125297464
Total jobs: 29

Toolchain	C++	Formatting	Contracts	Build	Modules	import_std	no_crtp	freestanding
Apple Clang 16.1	20	std::format	gsl-lite	Debug	—	—	—	—
Apple Clang 16.1	23	std::format	gsl-lite	Debug	—	—	—	—
Clang-16 (x86-64)	20	fmtlib	gsl-lite	Debug	—	—	—	—
Clang-16 (x86-64)	20	fmtlib	ms-gsl	Release	—	—	—	—
Clang-17 (x86-64)	20	std::format	gsl-lite	Debug	—	—	—	—
Clang-17 (x86-64)	20	std::format	gsl-lite	Debug	■	—	—	—
Clang-18 (arm64)	20	std::format	gsl-lite	Debug	—	—	—	—
Clang-18 (arm64)	20	std::format	gsl-lite	Debug	■	—	—	—
Clang-18 (arm64)	23	fmtlib	gsl-lite	Release	■	—	■	—
Clang-18 (x86-64)	20	fmtlib	ms-gsl	Release	■	—	—	—
Clang-18 (x86-64)	20	std::format	gsl-lite	Debug	—	—	—	—
Clang-18 (x86-64)	20	std::format	gsl-lite	Debug	■	—	—	—
Clang-18 (x86-64)	23	std::format	none	Release	■	■	■	—
Clang-20 (x86-64)	20	std::format	gsl-lite	Debug	—	—	—	—
Clang-20 (x86-64)	20	std::format	gsl-lite	Debug	■	—	—	—
Clang-20 (x86-64)	23	std::format	none	Debug	■	■	■	—
Clang-21 (x86-64)	20	std::format	gsl-lite	Debug	—	—	—	—
Clang-21 (x86-64)	20	std::format	gsl-lite	Debug	■	—	—	—
Clang-21 (x86-64)	23	std::format	none	Debug	■	■	■	—
GCC-12	20	fmtlib	gsl-lite	Debug	—	—	—	—
GCC-12	20	fmtlib	ms-gsl	Release	—	—	—	—
GCC-13	20	std::format	gsl-lite	Debug	—	—	—	—
GCC-13	23	std::format	gsl-lite	Release	—	—	—	—
GCC-14	20	std::format	gsl-lite	Debug	—	—	—	—
GCC-14	23	fmtlib	none	Debug	—	—	■	—
GCC-15	20	std::format	gsl-lite	Debug	—	—	—	—
GCC-15	23	fmtlib	gsl-lite	Debug	—	—	■	—
MSVC 14.4	20	std::format	gsl-lite	Debug	—	—	—	—
MSVC 14.4	23	std::format	none	Release	—	—	■	—

EACH VECTOR CATCHES DIFFERENT BUGS

- **Modules:** Missing **export**
- **import std:::** Header/module mixing conflicts
- **Freestanding:** Accidental **<iostream>**
- **gsl-lite vs ms-gsl:** Different macros
- **fmtlib vs std::format:** API differences
- **Oldest compiler:** Too-new features

Fuzz Your Build Matrix

Generate build matrix for Conan CI summary

Build Matrix

Random seed: 1125297464
Total jobs: 29

Toolchain	C++	Formatting	Contracts	Build	Modules	import_std	no_crtp	freestanding
Apple Clang 16.1	20	std::format	gsl-lite	Debug	—	—	—	—
Apple Clang 16.1	23	std::format	gsl-lite	Debug	—	—	—	—
Clang-16 (x86-64)	20	fmtlib	gsl-lite	Debug	—	—	—	—
Clang-16 (x86-64)	20	fmtlib	ms-gsl	Release	—	—	—	—
Clang-17 (x86-64)	20	std::format	gsl-lite	Debug	—	—	—	—
Clang-17 (x86-64)	20	std::format	gsl-lite	Debug	■	—	—	—
Clang-18 (arm64)	20	std::format	gsl-lite	Debug	—	—	—	—
Clang-18 (arm64)	20	std::format	gsl-lite	Debug	■	—	—	—
Clang-18 (arm64)	23	fmtlib	gsl-lite	Release	■	—	■	—
Clang-18 (x86-64)	20	fmtlib	ms-gsl	Release	■	—	—	—
Clang-18 (x86-64)	20	std::format	gsl-lite	Debug	—	—	—	—
Clang-18 (x86-64)	20	std::format	gsl-lite	Debug	■	—	—	—
Clang-18 (x86-64)	23	std::format	none	Release	■	■	■	—
Clang-20 (x86-64)	20	std::format	gsl-lite	Debug	—	—	—	—
Clang-20 (x86-64)	20	std::format	gsl-lite	Debug	■	—	—	—
Clang-20 (x86-64)	23	std::format	none	Debug	■	■	■	—
Clang-21 (x86-64)	20	std::format	gsl-lite	Debug	—	—	—	—
Clang-21 (x86-64)	20	std::format	gsl-lite	Debug	■	—	—	—
Clang-21 (x86-64)	23	std::format	none	Debug	■	■	■	—
GCC-12	20	fmtlib	gsl-lite	Debug	—	—	—	—
GCC-12	20	fmtlib	ms-gsl	Release	—	—	—	—
GCC-13	20	std::format	gsl-lite	Debug	—	—	—	—
GCC-13	23	std::format	gsl-lite	Release	—	—	—	—
GCC-14	20	std::format	gsl-lite	Debug	—	—	—	—
GCC-14	23	fmtlib	none	Debug	—	—	■	—
GCC-15	20	std::format	gsl-lite	Debug	—	—	—	—
GCC-15	23	fmtlib	gsl-lite	Debug	—	—	■	—
MSVC 14.4	20	std::format	gsl-lite	Debug	—	—	—	—
MSVC 14.4	23	std::format	none	Release	—	—	■	—

EACH VECTOR CATCHES DIFFERENT BUGS

- **Modules:** Missing **export**
- **import std:::** Header/module mixing conflicts
- **Freestanding:** Accidental **<iostream>**
- **gsl-lite vs ms-gsl:** Different macros
- **fmtlib vs std::format:** API differences
- **Oldest compiler:** Too-new features

Random seed: 100%
reproduction possible.

Fuzz Your Build Matrix

Generate build matrix for Conan CI summary

Build Matrix

Random seed: 1125297464
Total jobs: 29

Toolchain	C++	Formatting	Contracts	Build	Modules	import_std	no_crtp	freestanding
Apple Clang 16.1	20	std-format	gsl-lite	Debug	—	—	—	—
Apple Clang 16.1	23	std-format	gsl-lite	Debug	—	—	—	—
Clang-16 (x86-64)	20	fmtlib	gsl-lite	Debug	—	—	—	—
Clang-16 (x86-64)	20	fmtlib	ms-gsl	Release	—	—	—	—
Clang-17 (x86-64)	20	std-format	gsl-lite	Debug	—	—	—	—
Clang-17 (x86-64)	20	std-format	gsl-lite	Debug	■	—	—	—
Clang-18 (arm64)	20	std-format	gsl-lite	Debug	—	—	—	—
Clang-18 (arm64)	20	std-format	gsl-lite	Debug	■	—	—	—
Clang-18 (arm64)	23	fmtlib	gsl-lite	Release	■	—	■	—
Clang-18 (x86-64)	20	fmtlib	ms-gsl	Release	■	—	—	—
Clang-18 (x86-64)	20	std-format	gsl-lite	Debug	—	—	—	—
Clang-18 (x86-64)	20	std-format	gsl-lite	Debug	■	—	—	—
Clang-18 (x86-64)	23	std-format	none	Release	■	■	■	—
Clang-20 (x86-64)	20	std-format	gsl-lite	Debug	—	—	—	—
Clang-20 (x86-64)	20	std-format	gsl-lite	Debug	■	—	—	—
Clang-20 (x86-64)	23	std-format	none	Debug	■	■	■	—
Clang-21 (x86-64)	20	std-format	gsl-lite	Debug	—	—	—	—
Clang-21 (x86-64)	20	std-format	gsl-lite	Debug	■	—	—	—
Clang-21 (x86-64)	23	std-format	none	Debug	■	■	■	—
GCC-12	20	fmtlib	gsl-lite	Debug	—	—	—	—
GCC-12	20	fmtlib	ms-gsl	Release	—	—	—	—
GCC-13	20	std-format	gsl-lite	Debug	—	—	—	—
GCC-13	23	std-format	gsl-lite	Release	—	—	—	—
GCC-14	20	std-format	gsl-lite	Debug	—	—	—	—
GCC-14	23	fmtlib	none	Debug	—	—	■	—
GCC-15	20	std-format	gsl-lite	Debug	—	—	—	—
GCC-15	23	fmtlib	gsl-lite	Debug	—	—	■	—
MSVC 14.4	20	std-format	gsl-lite	Debug	—	—	—	—
MSVC 14.4	23	std-format	none	Release	—	—	■	—

▼ Coverage statistics

```
toolchain=Apple Clang 16.1: 2
toolchain=Clang-16 (x86-64): 2
toolchain=Clang-17 (x86-64): 2
toolchain=Clang-18 (arm64): 3
toolchain=Clang-18 (x86-64): 4
toolchain=Clang-20 (x86-64): 3
toolchain=Clang-21 (x86-64): 3
toolchain=GCC-12: 2
toolchain=GCC-13: 2
toolchain=GCC-14: 2
toolchain=GCC-15: 2
toolchain=MSVC 14.4: 2
std=20: 20
std=23: 9
build_type=Debug: 22
build_type=Release: 7
contracts=gsl-lite: 21
contracts=ms-gsl: 3
contracts=none: 5
cxx_modules=False: 19
cxx_modules=True: 10
std_format=False: 8
std_format=True: 21
import_std=False: 26
import_std=True: 3
no_crtp=False: 22
no_crtp=True: 7
freestanding=False: 29
```

Fuzz Your Build Matrix

Generate build matrix for Conan CI summary

Build Matrix

Random seed: 1125297464
Total jobs: 29

Toolchain	C++	Formatting	Contracts	Build	Modules	import_std	no_crtp	freestanding
Apple Clang 16.1	20	std-format	gsl-lite	Debug	—	—	—	—
Apple Clang 16.1	23	std-format	gsl-lite	Debug	—	—	—	—
Clang-16 (x86-64)	20	fmtlib	gsl-lite	Debug	—	—	—	—
Clang-16 (x86-64)	20	fmtlib	ms-gsl	Release	—	—	—	—
Clang-17 (x86-64)	20	std-format	gsl-lite	Debug	—	—	—	—
Clang-17 (x86-64)	20	std-format	gsl-lite	Debug	■	—	—	—
Clang-18 (arm64)	20	std-format	gsl-lite	Debug	—	—	—	—
Clang-18 (arm64)	20	std-format	gsl-lite	Debug	■	—	—	—
Clang-18 (arm64)	23	fmtlib	gsl-lite	Release	■	—	■	—
Clang-18 (x86-64)	20	fmtlib	ms-gsl	Release	■	—	—	—
Clang-18 (x86-64)	20	std-format	gsl-lite	Debug	—	—	—	—
Clang-18 (x86-64)	20	std-format	gsl-lite	Debug	■	—	—	—
Clang-18 (x86-64)	23	std-format	none	Release	■	■	■	—
Clang-20 (x86-64)	20	std-format	gsl-lite	Debug	—	—	—	—
Clang-20 (x86-64)	20	std-format	gsl-lite	Debug	■	—	—	—
Clang-20 (x86-64)	23	std-format	none	Debug	■	■	■	—
Clang-21 (x86-64)	20	std-format	gsl-lite	Debug	—	—	—	—
Clang-21 (x86-64)	20	std-format	gsl-lite	Debug	■	—	—	—
Clang-21 (x86-64)	23	std-format	none	Debug	■	■	■	—
GCC-12	20	fmtlib	gsl-lite	Debug	—	—	—	—
GCC-12	20	fmtlib	ms-gsl	Release	—	—	—	—
GCC-13	20	std-format	gsl-lite	Debug	—	—	—	—
GCC-13	23	std-format	gsl-lite	Release	—	—	—	—

▼ Coverage statistics

```
toolchain=Apple Clang 16.1: 2
toolchain=Clang-16 (x86-64): 2
toolchain=Clang-17 (x86-64): 2
toolchain=Clang-18 (arm64): 3
toolchain=Clang-18 (x86-64): 4
toolchain=Clang-20 (x86-64): 3
toolchain=Clang-21 (x86-64): 3
toolchain=GCC-12: 2
toolchain=GCC-13: 2
toolchain=GCC-14: 2
toolchain=GCC-15: 2
toolchain=MSVC 14.4: 2
std=20: 20
std=23: 9
build_type=Debug: 22
build_type=Release: 7
contracts=gsl-lite: 21
contracts=ms-gsl: 3
contracts=none: 5
cxx_modules=False: 19
cxx_modules=True: 10
std_format=False: 8
std_format=True: 21
import_std=False: 26
```

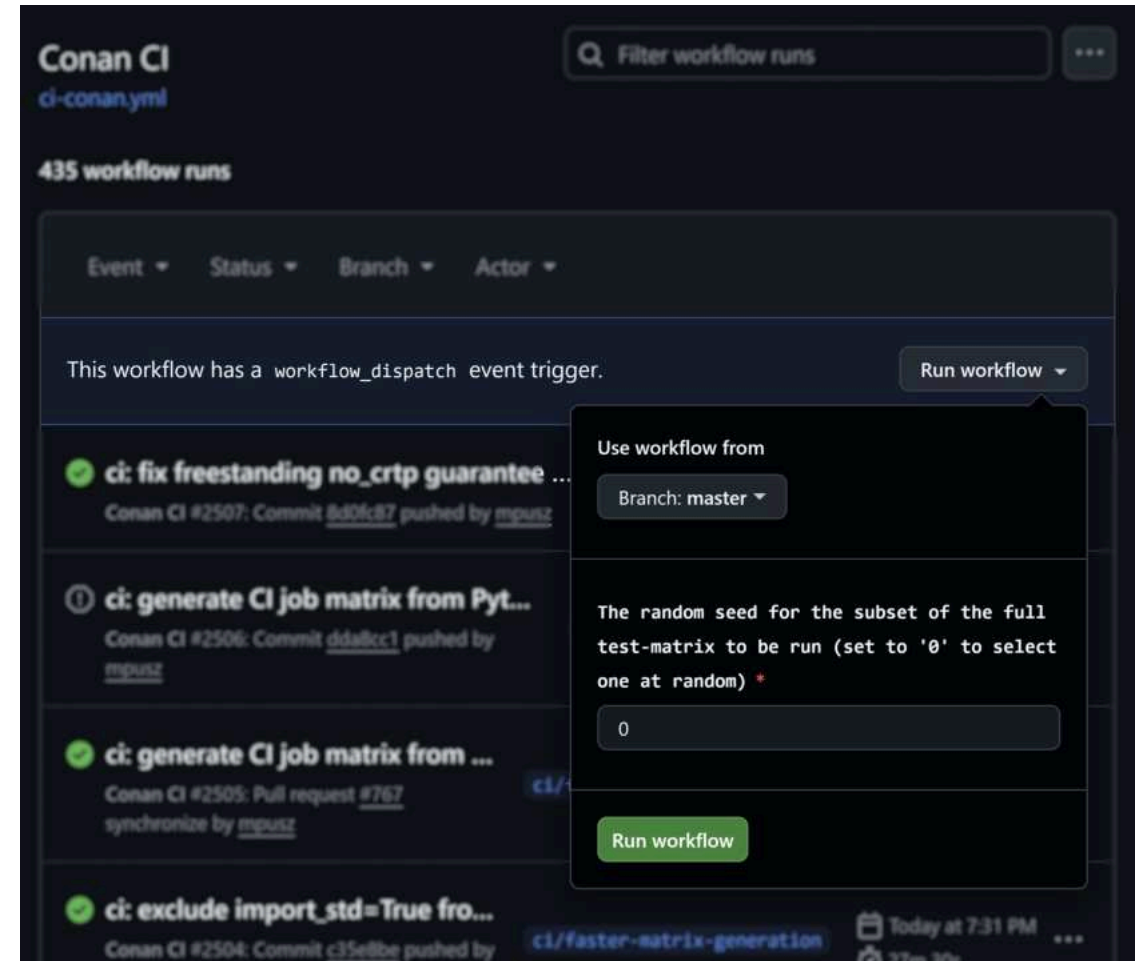
Minimum coverage guarantee: Each config option appears at least twice.

Random, but Reproducible

- **Random seed:** GitHub Actions

`workflow_dispatch` input

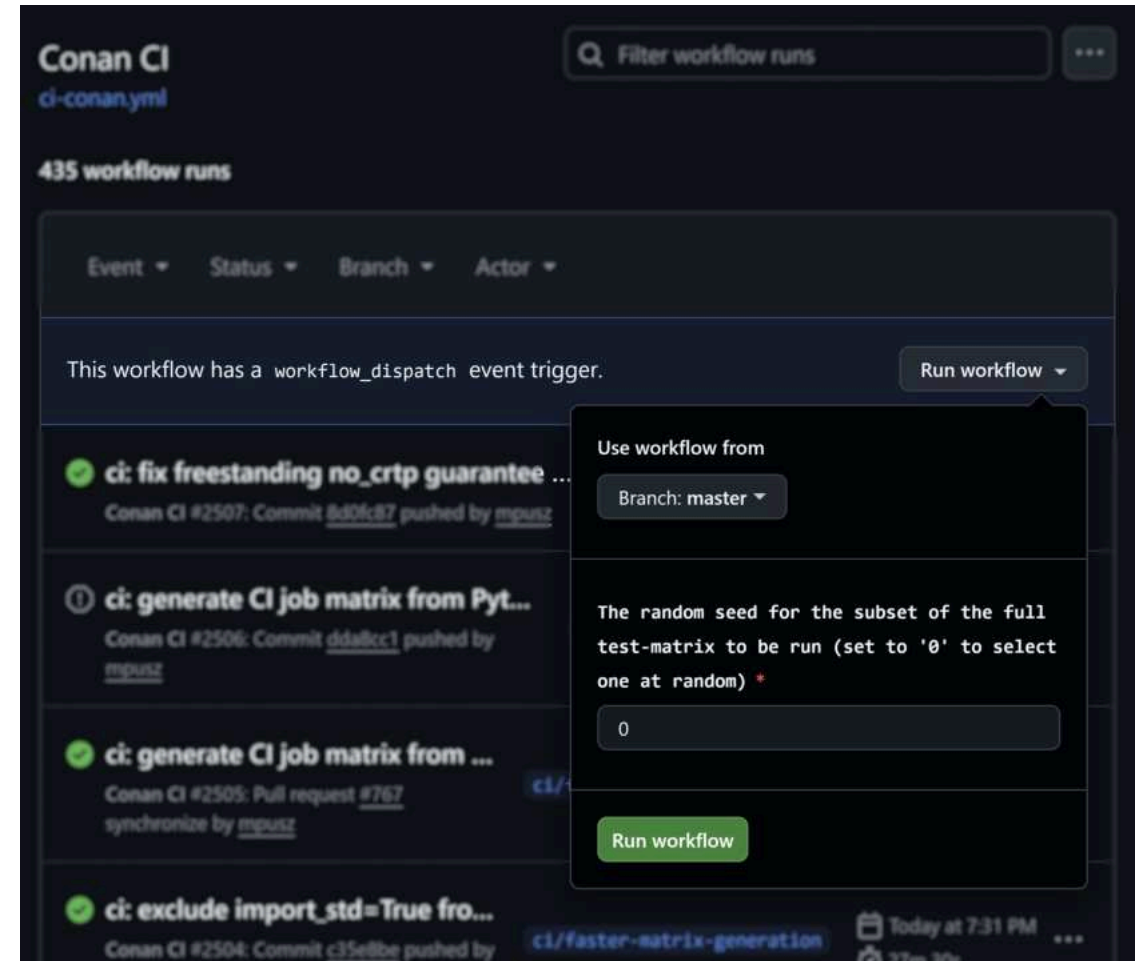
- *Specify seed* → reproduce exact failing configuration
- *Seed = 0* → generate new randomized matrix (printed in logs)



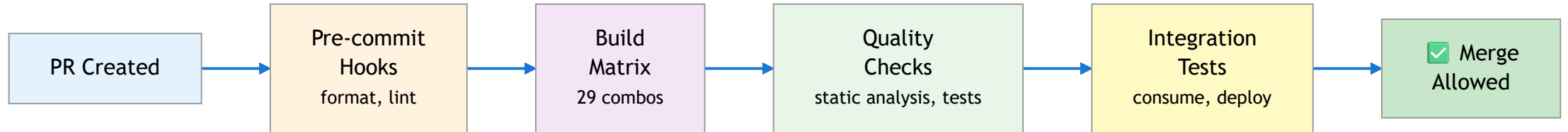
Random, but Reproducible

- **Random seed:** GitHub Actions
`workflow_dispatch` input
 - *Specify seed* → reproduce exact failing configuration
 - *Seed = 0* → generate new randomized matrix (printed in logs)

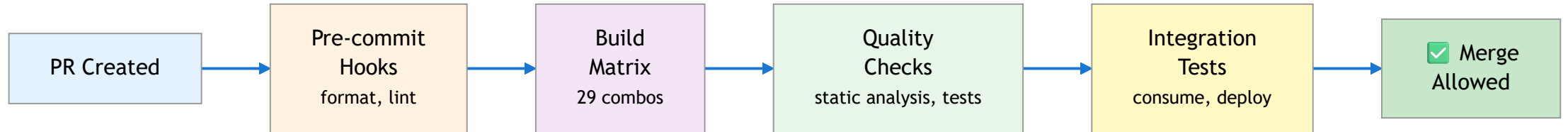
Result: Broad coverage without combinatorial explosion.



The Quality Gate - Complete Flow



The Quality Gate - Complete Flow



The Complete Defense:

- 🛡️ **6 layers** of validation before merge
- 🚫 **No single point of failure** - each layer catches different issues
- ✅ **High confidence** - if all green, it's production-ready
- 🎯 **Trust earned** - users see "all checks passed" and believe it

What "All Checks Passed" Means to Users?

LOWER RISK

- **Fewer surprises** - Edge cases caught before you hit them
- **Faster debugging** - Issues reproduced quickly with same matrix
- **Better support** - Maintainer can validate your bug reports



TIP 3

COLLABORATIVE TRIAGE

Collaborative Triage

Communication IS a feature of your library.

Collaborative Triage

Communication IS a feature of your library.

How you treat an issue defines your community.

Separate Issues from Discussions

ISSUES

- Actionable bugs
- Tasks
- Specific Feature Requests
- Security vulnerabilities

Separate Issues from Discussions

ISSUES

- Actionable bugs
- Tasks
- Specific Feature Requests
- Security vulnerabilities

DISCUSSIONS

- "How do I...?" (Q&A)
- Show and Tell
- Ideas
- Design Proposals / RFCs
- Brainstorming

Separate Issues from Discussions

ISSUES

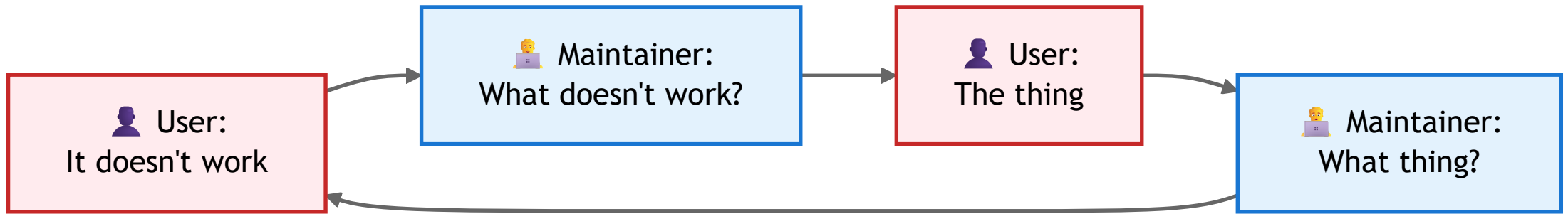
- Actionable bugs
- Tasks
- Specific Feature Requests
- Security vulnerabilities

DISCUSSIONS

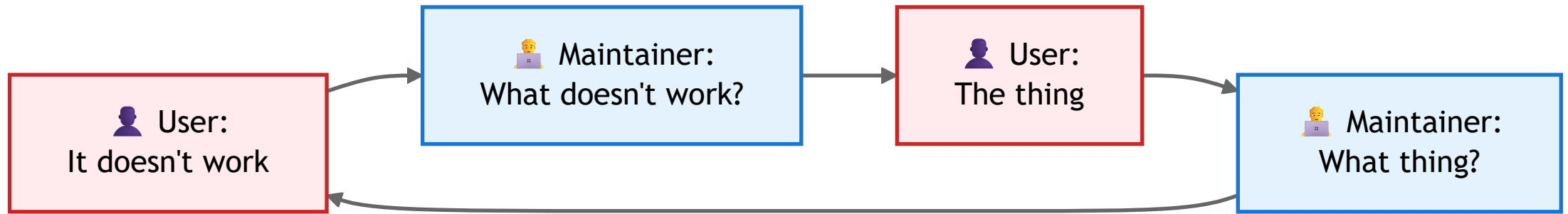
- "How do I...?" (Q&A)
- Show and Tell
- Ideas
- Design Proposals / RFCs
- Brainstorming

Enable GitHub Discussions - it's free.

The "It Doesn't Work" Loop



The "It Doesn't Work" Loop

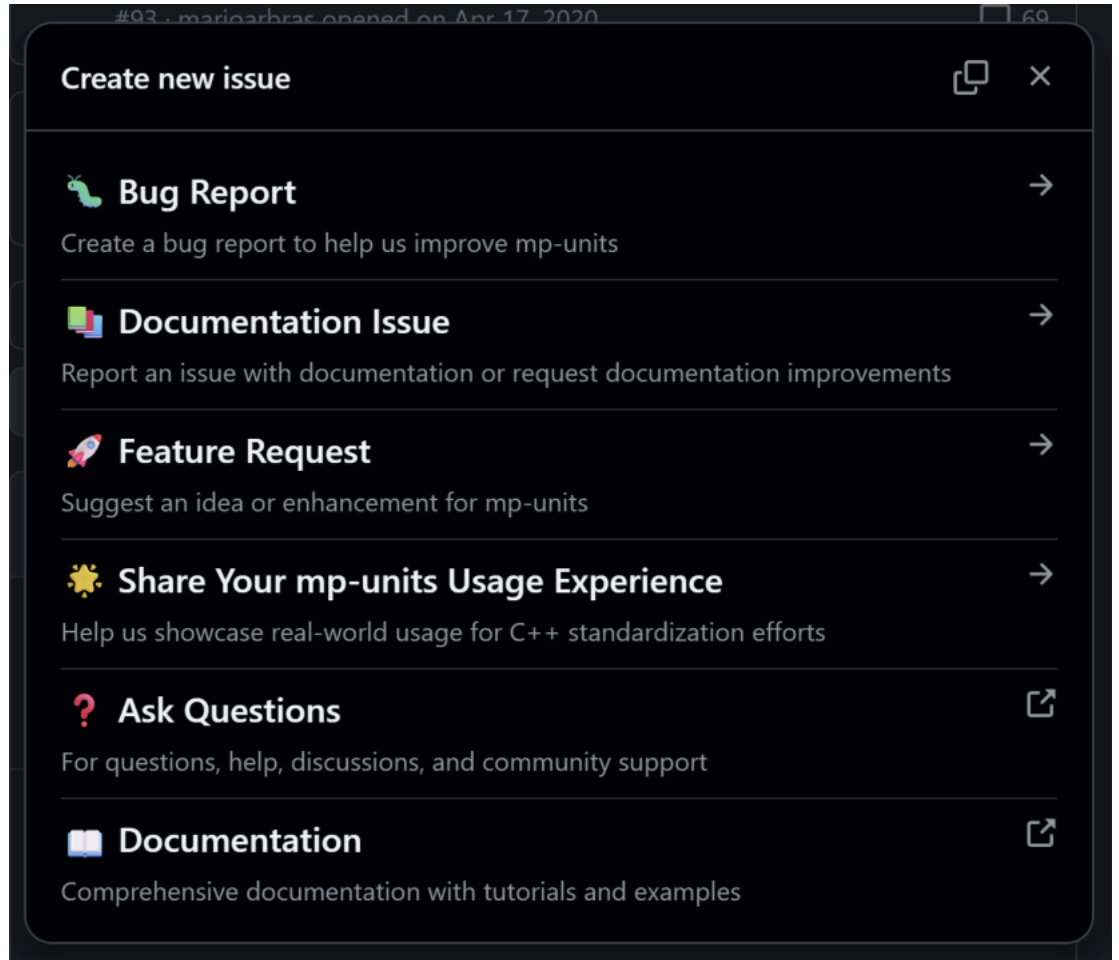


This wastes everyone's time — Stop it with templates!

Issue Templates

REQUEST

- Description
- Steps to reproduce
- **Compiler Explorer link**
- Build config
 - OS
 - Compiler
 - C++ version
 - product version
 - product features enabled



TIP 4

DOCUMENTATION AS PROSE

The Documentation Paradox

How many of you enjoy writing documentation?

The Documentation Paradox

How many of you enjoy writing documentation?

How many of you appreciate good documentation in the 3rd Party Software?

"We Have Plenty of Documentation!"



README.md Is Not Enough

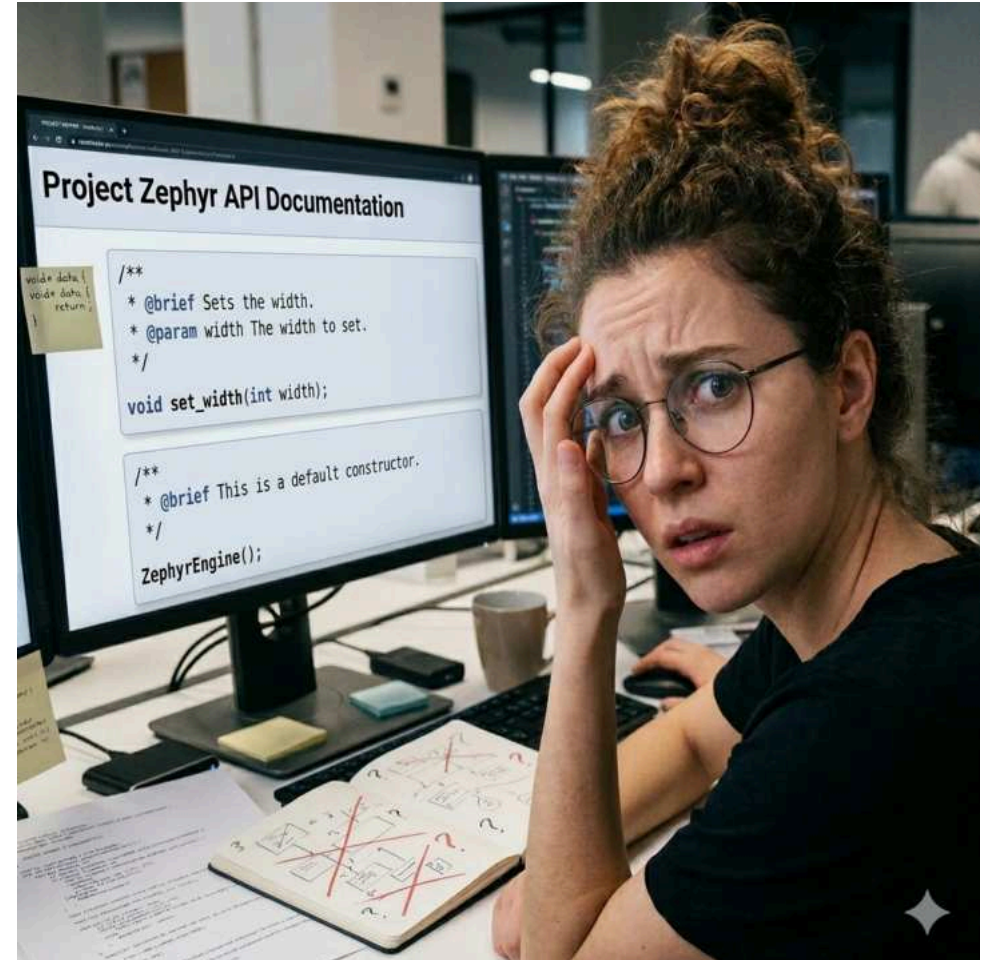
Lots of C++ projects do not provide any other documentation besides README.md.

README.md Is Not Enough

Lots of C++ projects do not provide any other documentation besides README.md.

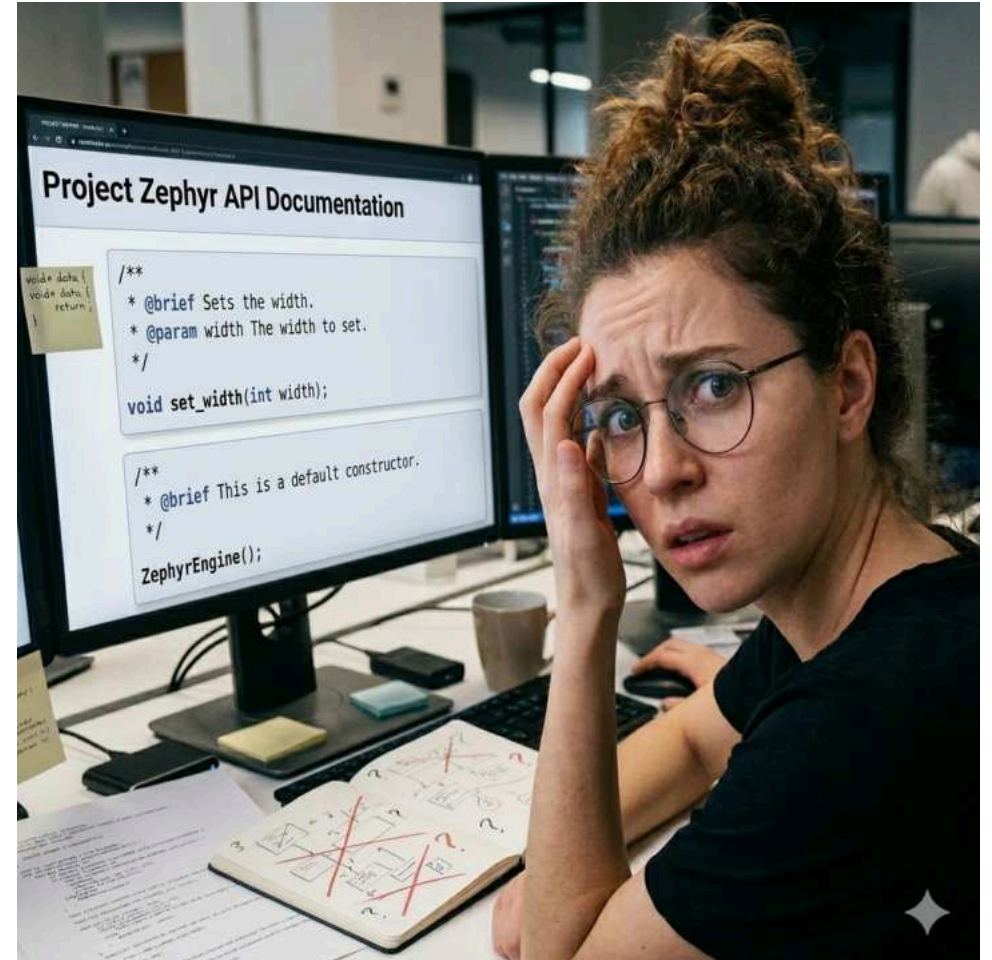
Let's just run Doxygen on the current code and publish the outcome... No!

The Doxygen Trap



The Doxygen Trap

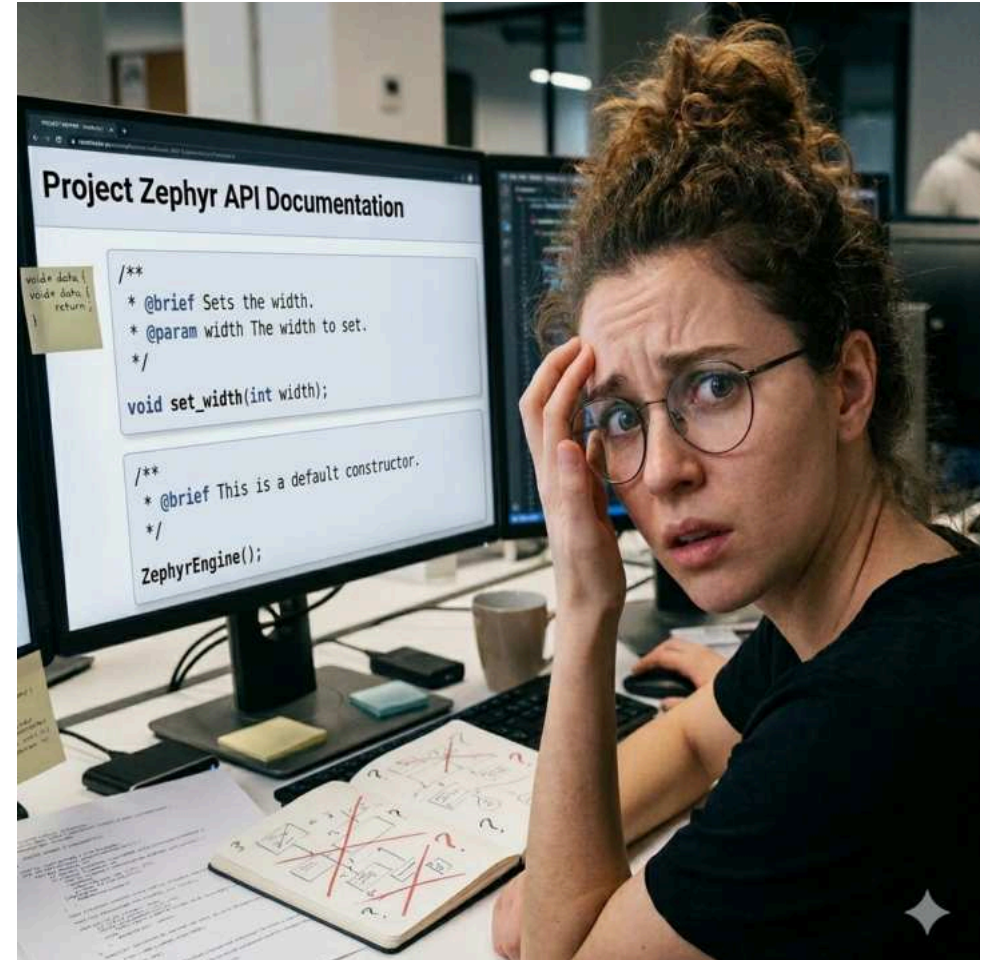
Doxygen encourages
"Inventory Documentation".



The Doxygen Trap

Doxygen encourages
"Inventory Documentation".

```
/**  
 * @brief Sets the width.  
 * @param width The width to set.  
 */  
void set_width(int width);
```



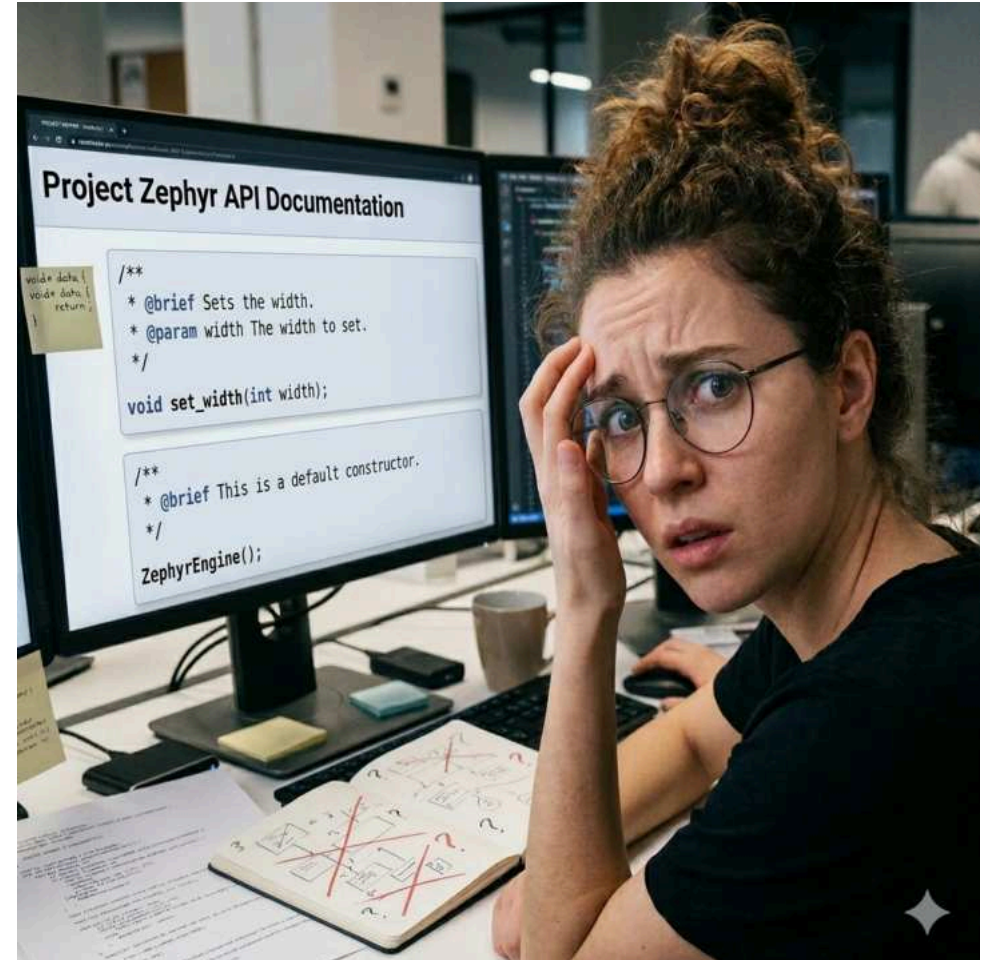
The Doxygen Trap

Doxygen encourages
"Inventory Documentation".

```
/**  
 * @brief Sets the width.  
 * @param width The width to set.  
 */  
void set_width(int width);
```

THE TAUTOLOGY PROBLEM

- Tells you *what* is there
- Doesn't explain *why* or *how* to use it
- 1,000 pages of docs, not helpful for new users



Reference vs. Tutorial



DOXYGEN

- A dictionary
- Great when you know the word you look for
- Useless for learning the framework
- Often auto-generated from raw code with no custom descriptions ☹️

Reference vs. Tutorial



DOXYGEN

- A dictionary
- Great when you know the word you look for
- Useless for learning the framework
- Often auto-generated from raw code with no custom descriptions ☹️



PROSE DOCUMENTATION

- A textbook
- Guides you through concepts
- Explains the architecture and rationale
- Provides a curated learning path

It is not boring at all

It is not boring at all

Writing documentation might be a fascinating design-related exercise!

It is not boring at all

Writing documentation might be a fascinating design-related exercise!

Show off 🎉

- You spent *weeks* on this feature
- Documentation is your stage

It is not boring at all

Writing documentation might be a fascinating design-related exercise!

Show off 🎉

- You spent *weeks* on this feature
- Documentation is your stage

Explain 🗨️

- Put yourself in the user's shoes
- Rethink the design from first principles
- Discover corner cases *before* users do

It is not boring at all

Writing documentation might be a fascinating design-related exercise!

Show off 🎉

- You spent *weeks* on this feature
- Documentation is your stage

Explain 🗨️

- Put yourself in the user's shoes
- Rethink the design from first principles
- Discover corner cases *before* users do

Improve 🛠️

- Writing prose reveals design flaws
- "How do I explain this?" → "Why is it this hard?"
- You *will* ship a better API

It is not boring at all

Writing documentation might be a fascinating design-related exercise!

Show off 🎉

- You spent *weeks* on this feature
- Documentation is your stage

Explain 🗨️

- Put yourself in the user's shoes
- Rethink the design from first principles
- Discover corner cases *before* users do

Improve 🛠️

- Writing prose reveals design flaws
- "How do I explain this?" → "Why is it this hard?"
- You *will* ship a better API

Repeat 🔁

- Design → Implement → Document → Redesign
- Each cycle tightens the product

The Diátaxis Framework

Systematic approach to technical documentation (diataxis.fr).

The Diátaxis Framework

Systematic approach to technical documentation (diataxis.fr).

QUADRANT	USER SAYS	FOCUS
 Tutorials	"I want to learn"	Learning-oriented
 How-To Guides	"I need to solve X"	Goal-oriented
 Explanation	"I want to understand"	Understanding-oriented
 Reference	"What's the API?"	Information-oriented

The Diátaxis Framework

Systematic approach to technical documentation (diataxis.fr).

QUADRANT	USER SAYS	FOCUS
 Tutorials	"I want to learn"	Learning-oriented
 How-To Guides	"I need to solve X"	Goal-oriented
 Explanation	"I want to understand"	Understanding-oriented
 Reference	"What's the API?"	Information-oriented

Most doxygen-based documentation address only the fourth quadrant.

mp-units Homepage

 **mp-units** HEAD 

 Search

 mpusz/mp-units
v2.5.0 1.4k 119

[Home](#) [Getting Started](#) [Tutorials](#) [User's Guide](#) [Workshops](#) [How-to Guides](#) [Examples](#) [Reference](#) [Blog](#) [Release Notes](#)

Welcome to mp-units!

mp-units is the only Modern C++ (C++20) library providing the full spectrum of compile-time safety for physical quantities and units – from dimensional analysis and unit conversions to **quantity kind safety** – built on the ISO 80000 International System of Quantities (ISQ).

The library source code is hosted on [GitHub](#) with a permissive [MIT license](#).

C++ modules

Header files

```
#include <iostream>
#include <print>
import mp_units;

using namespace mp_units;

inline constexpr struct smoot final : named_unit<"smoot", mag<67> * usc::inch> {} smoot;

int main()
{
    constexpr quantity dist = 364.4 * smoot;
    std::println("Harvard Bridge length = {:.1f} ({:N[.1f]}, {:N[.2f]}) ± 1 ear",
                dist, dist.in(usc::foot), dist.in(si::metre));
}
```

Output:

```
Harvard Bridge length = 364.4 smoot (2834.6 ft, 620.14 m) ± 1 ear
```

Try it live on Compiler Explorer

mp-units Navigation: Diátaxis in Action

 **mp-units** HEAD ▾

  Search

 mpusz/mp-units
v2.5.0 1.4k 119

Home Getting Started **Tutorials** User's Guide Workshops How-to Guides Examples Reference Blog Release Notes

Welcome to mp-units!

mp-units is the only Modern C++ (C++20) library providing the full spectrum of compile-time safety for physical quantities and units — unit conversions to **quantity kind safety** — built on the ISO 80000 International System of Quantities (ISQ).

The library source code is hosted on [GitHub](#) with a permissive [MIT license](#).

C++ modules Header files

```
#include <iostream>
#include <print>
import mp_units;

using namespace mp_units;

inline constexpr struct smoot final : named_unit<"smoot", mag<67> * usc::inch> {} smoot;

int main()
{
    constexpr quantity dist = 364.4 * smoot;
    std::println("Harvard Bridge length = {:.1f} ({:N[.1f]}, {:N[.2f]}) ± 1 ear",
                dist, dist.in(usc::foot), dist.in(si::metre));
}
```

Output:

```
Harvard Bridge length = 364.4 smoot (2834.6 ft, 620.14 m) ± 1 ear
```

Try it live on Compiler Explorer

Diátaxis Categories

-  Tutorials (Learning)
-  How-To (Goal)
-  Explanation
-  Reference
-  Workshops (Hybrid)
-  Beyond Diátaxis
-  Home/Welcome

mp-units Navigation: Diátaxis in Action

The screenshot displays the mp-units website with a dark blue header. The header includes the mp-units logo, a 'HEAD' dropdown, a search bar, and repository statistics for 'mpusz/mp-units' (v2.5.0, 1.4k stars, 119 forks). A navigation bar below the header contains links: Home, Getting Started, Tutorials, User's Guide, Workshops, How-to Guides, Examples, Reference, Blog, and Release Notes. The main content area starts with a 'Welcome to mp-units!' message, followed by a description of the library as a Modern C++ (C++20) library for physical quantities and units, emphasizing 'quantity kind safety' and its foundation on the ISO 80000 International System of Quantities (ISQ). It also mentions that the source code is on GitHub with a MIT license. Below this is a code editor showing C++ modules and header files. The code includes `<iostream>`, `<print>`, and `mp_units`, defines a `smoot` unit, and prints the length of the Harvard Bridge in `smoot`, `foot`, and `metre`. The output shows the bridge length as 364.4 smoot (2834.6 ft, 620.14 m) ± 1 ear. A button 'Try it live on Compiler Explorer' is at the bottom. A light blue box on the right, titled 'Diátaxis Categories', lists: Tutorials (Learning) with a blue circle, How-To (Goal) with a green circle, Explanation with a yellow circle, Reference with a red circle, Workshops (Hybrid) with a purple circle, Beyond Diátaxis with a grey circle, and Home/Welcome with a white circle.

Welcome to mp-units!

mp-units is the only Modern C++ (C++20) library providing the full spectrum of compile-time safety for physical quantities and units — unit conversions to **quantity kind safety** — built on the ISO 80000 International System of Quantities (ISQ).

The library source code is hosted on [GitHub](#) with a permissive [MIT license](#).

C++ modules Header files

```
#include <iostream>
#include <print>
import mp_units;

using namespace mp_units;

inline constexpr struct smoot final : named_unit<"smoot", mag<67> * usc::inch> {} smoot;

int main()
{
    constexpr quantity dist = 364.4 * smoot;
    std::println("Harvard Bridge length = {:.1f} ({:N[.1f]}, {:N[.2f]}) ± 1 ear",
                dist, dist.in(usc::foot), dist.in(si::metre));
}
```

Output:

```
Harvard Bridge length = 364.4 smoot (2834.6 ft, 620.14 m) ± 1 ear
```

[Try it live on Compiler Explorer](#)

Diátaxis Categories

- Tutorials (Learning)
- How-To (Goal)
- Explanation
- Reference
- Workshops (Hybrid)
- Beyond Diátaxis
- Home/Welcome

mp-units Navigation: Diátaxis in Action

The screenshot displays the mp-units website with a dark blue header. The header includes the mp-units logo, a 'HEAD' dropdown, a search bar, and repository statistics for 'mpusz/mp-units' (v2.5.0, 1.4k stars, 119 forks). A navigation bar below the header contains links: Home, Getting Started, Tutorials, User's Guide, Workshops, How-to Guides, Examples, Reference, Blog, and Release Notes. The main content area starts with a 'Welcome to mp-units!' message, followed by a description of the library as a Modern C++ (C++20) library for physical quantities and units, emphasizing 'quantity kind safety' and its foundation on the ISO 80000 International System of Quantities (ISQ). It also mentions that the source code is on GitHub with a MIT license. Below this is a code editor showing C++ modules and header files. The code includes `<iostream>`, `<print>`, and `mp_units`, defines a namespace `mp_units`, and shows a `main` function that calculates the Harvard Bridge length in smoots, feet, and meters. The output of the code is displayed in a box: 'Harvard Bridge length = 364.4 smoot (2834.6 ft, 620.14 m) ± 1 ear'. A button 'Try it live on Compiler Explorer' is at the bottom. On the right side, a light blue box titled 'Diátaxis Categories' lists seven categories with corresponding colored circles: Tutorials (Learning) in blue, How-To (Goal) in green, Explanation in yellow, Reference in red, Workshops (Hybrid) in purple, Beyond Diátaxis in grey, and Home/Welcome in white with a black outline.

Welcome to mp-units!

mp-units is the only Modern C++ (C++20) library providing the full spectrum of compile-time safety for physical quantities and units — unit conversions to **quantity kind safety** — built on the ISO 80000 International System of Quantities (ISQ).

The library source code is hosted on [GitHub](#) with a permissive [MIT license](#).

C++ modules Header files

```
#include <iostream>
#include <print>
import mp_units;

using namespace mp_units;

inline constexpr struct smoot final : named_unit<"smoot", mag<67> * usc::inch> {} smoot;

int main()
{
    constexpr quantity dist = 364.4 * smoot;
    std::println("Harvard Bridge length = {:.N[.1f]} ( {:.N[.1f]}, {:.N[.2f]}) ± 1 ear",
                dist, dist.in(usc::foot), dist.in(si::metre));
}
```

Output:

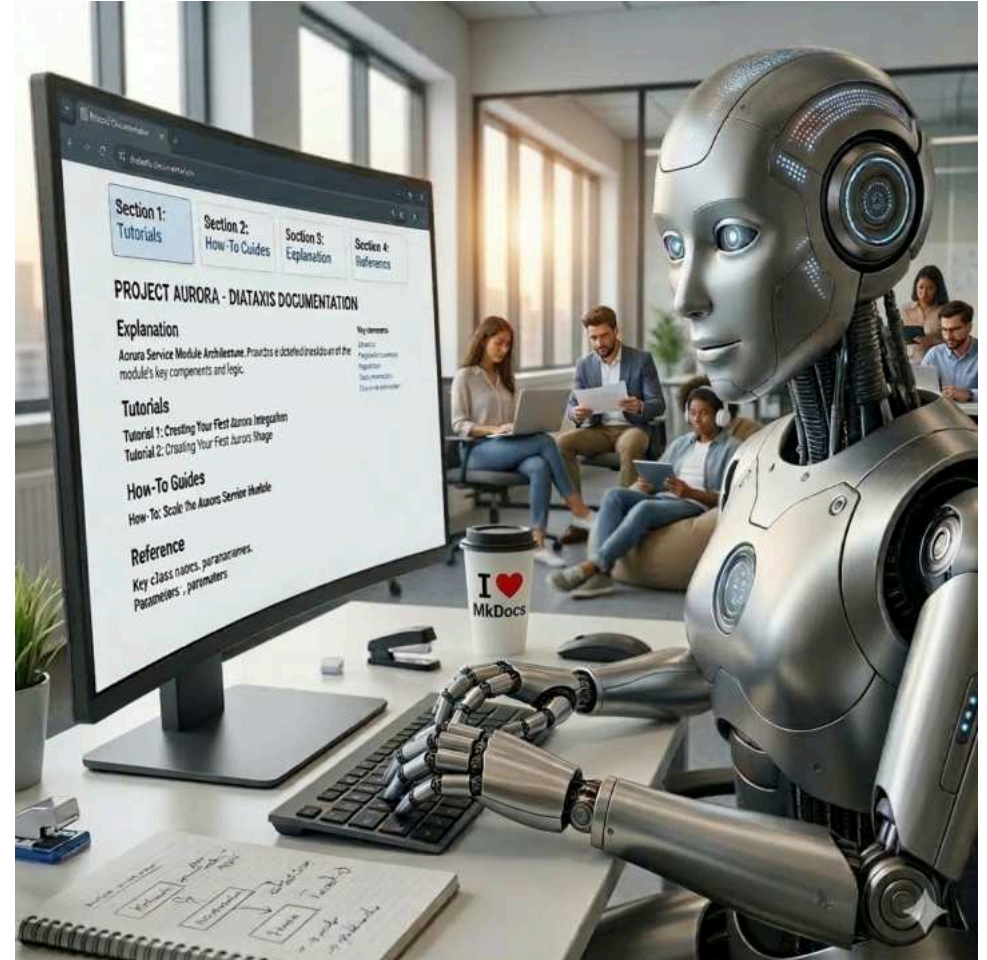
```
Harvard Bridge length = 364.4 smoot (2834.6 ft, 620.14 m) ± 1 ear
```

[Try it live on Compiler Explorer](#)

Diátaxis Categories

- Tutorials (Learning)
- How-To (Goal)
- Explanation
- Reference
- Workshops (Hybrid)
- Beyond Diátaxis
- Home/Welcome

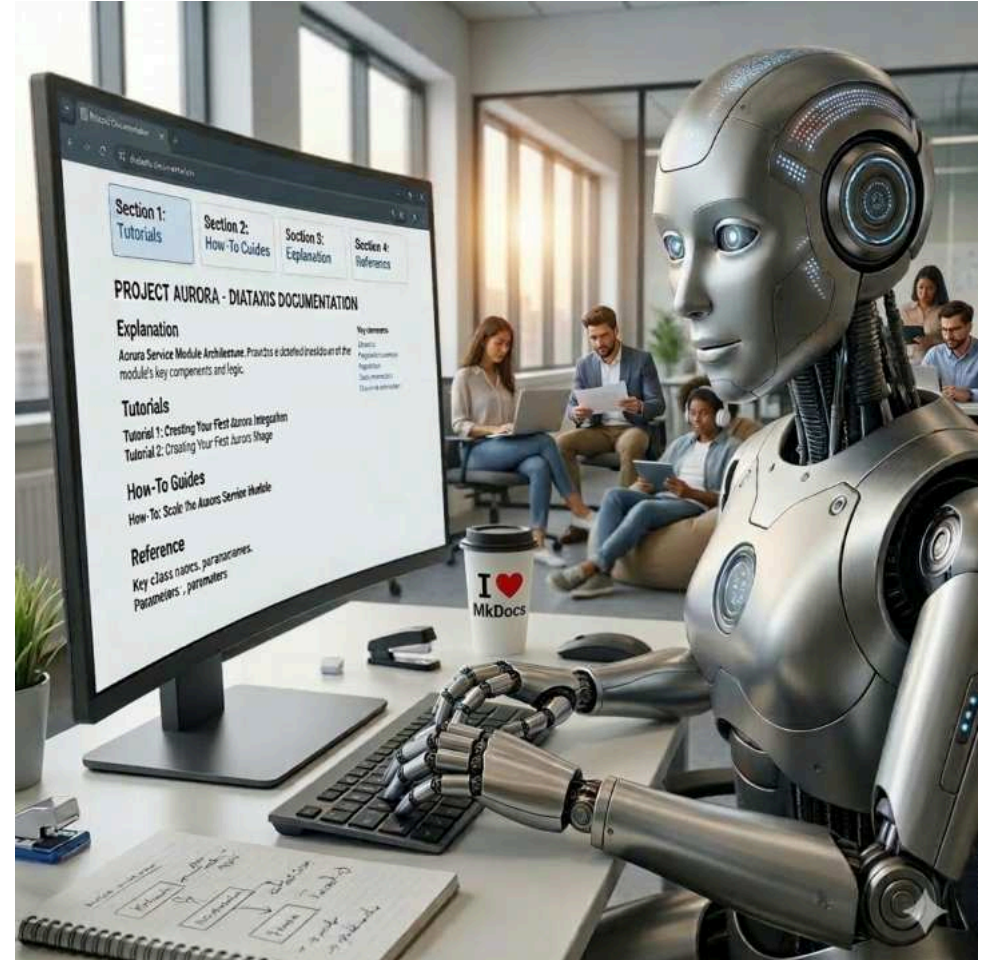
LLMs as Your Documentation Intern



LLMs as Your Documentation Intern

THE WORKFLOW

- *Feed* C++ headers + design notes to an LLM
- Clearly *describe* what you need
- **Human review** for technical accuracy



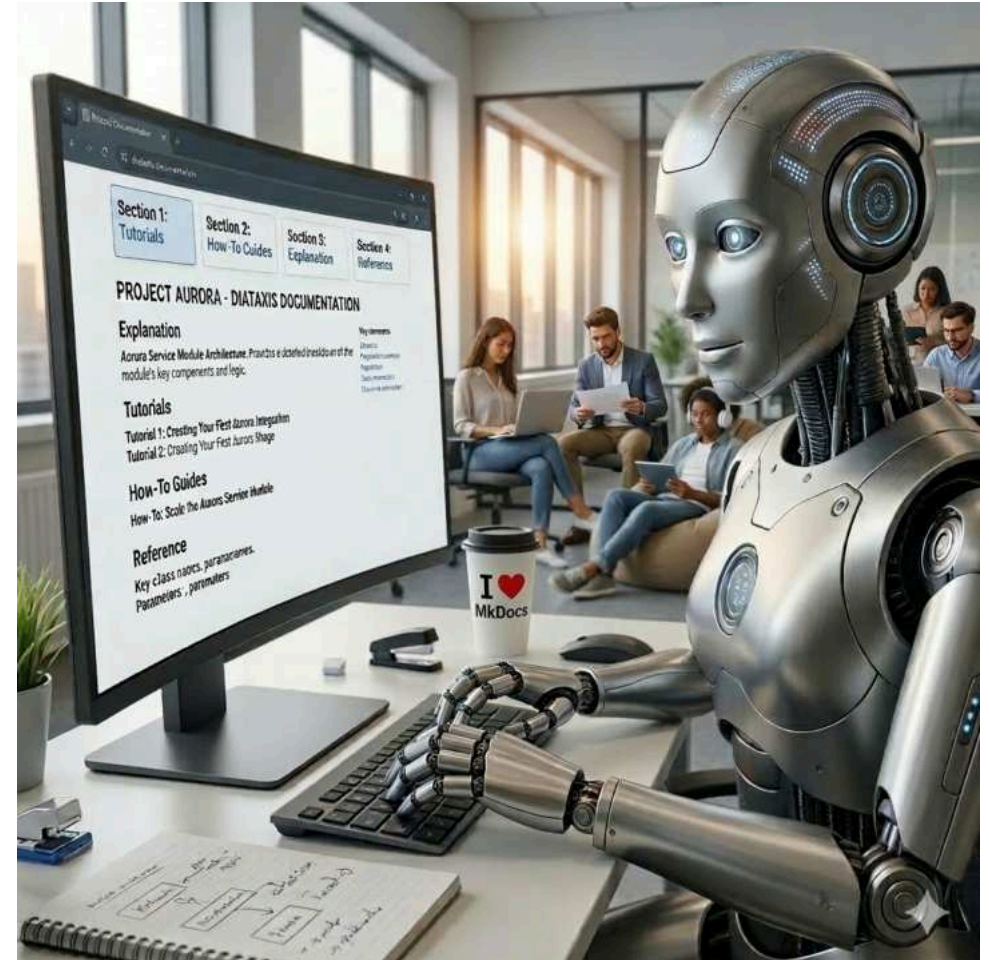
LLMs as Your Documentation Intern

THE WORKFLOW

- *Feed* C++ headers + design notes to an LLM
- Clearly *describe* what you need
- **Human review** for technical accuracy

BENEFIT

- *10 minutes* instead of 2 hours per page



LLMs as Your Documentation Intern

THE WORKFLOW

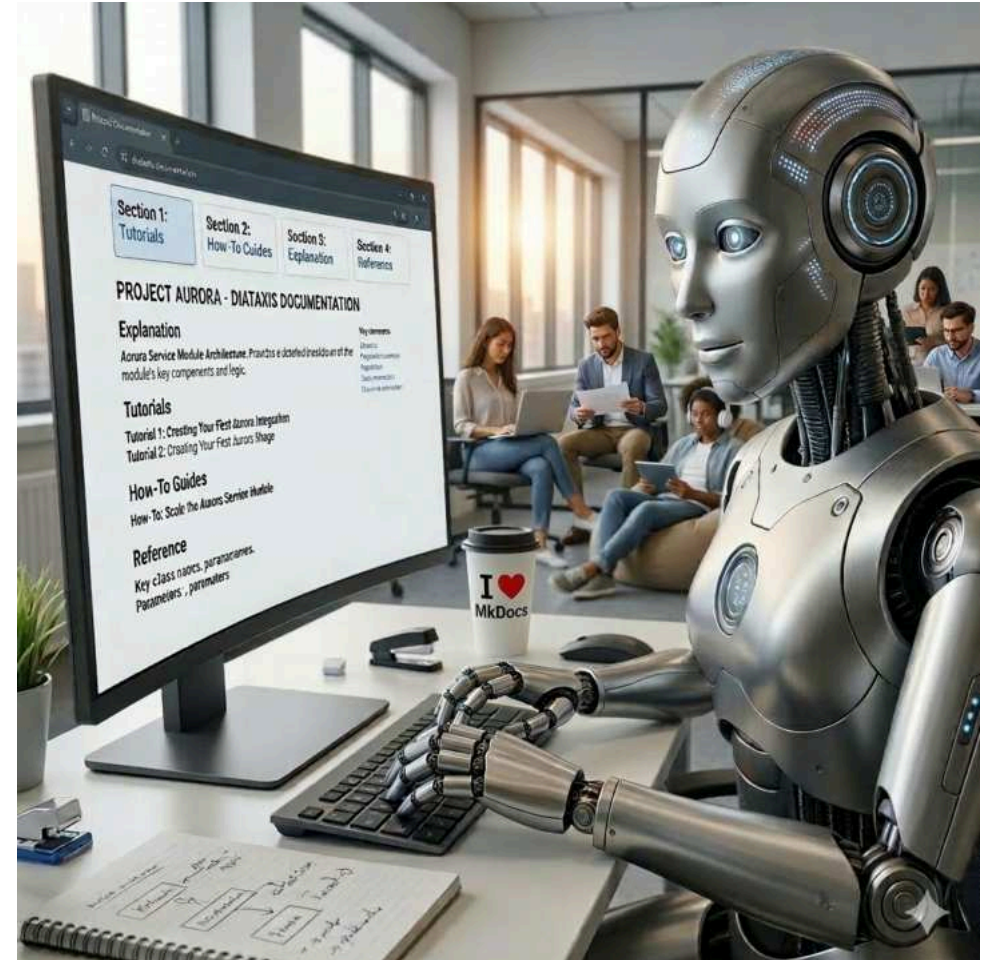
- *Feed* C++ headers + design notes to an LLM
- Clearly *describe* what you need
- **Human review** for technical accuracy

BENEFIT

- *10 minutes* instead of 2 hours per page

LLMS ARE GREAT WITH

- Text generation and *prose polishing*
- *Simple reasoning* and examples
- Converting bullet points to flowing text

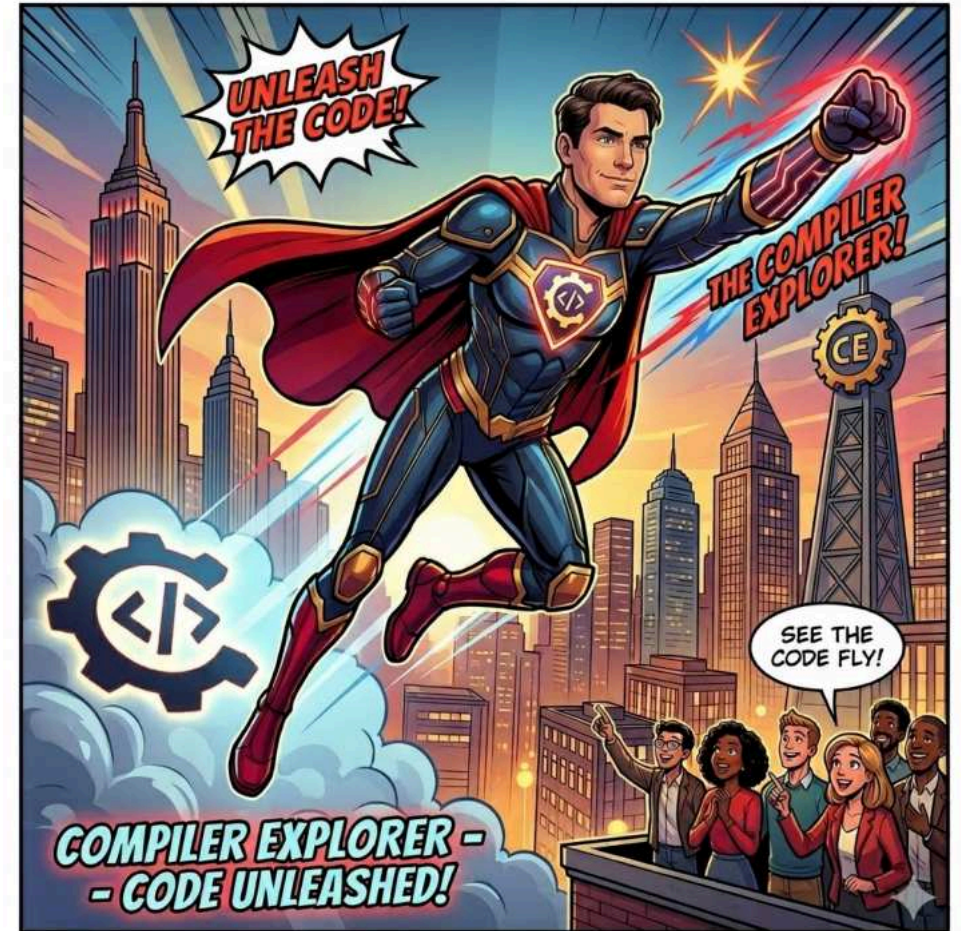


TIP 5

ZERO-INSTALL PROTOTYPING

Compiler Explorer

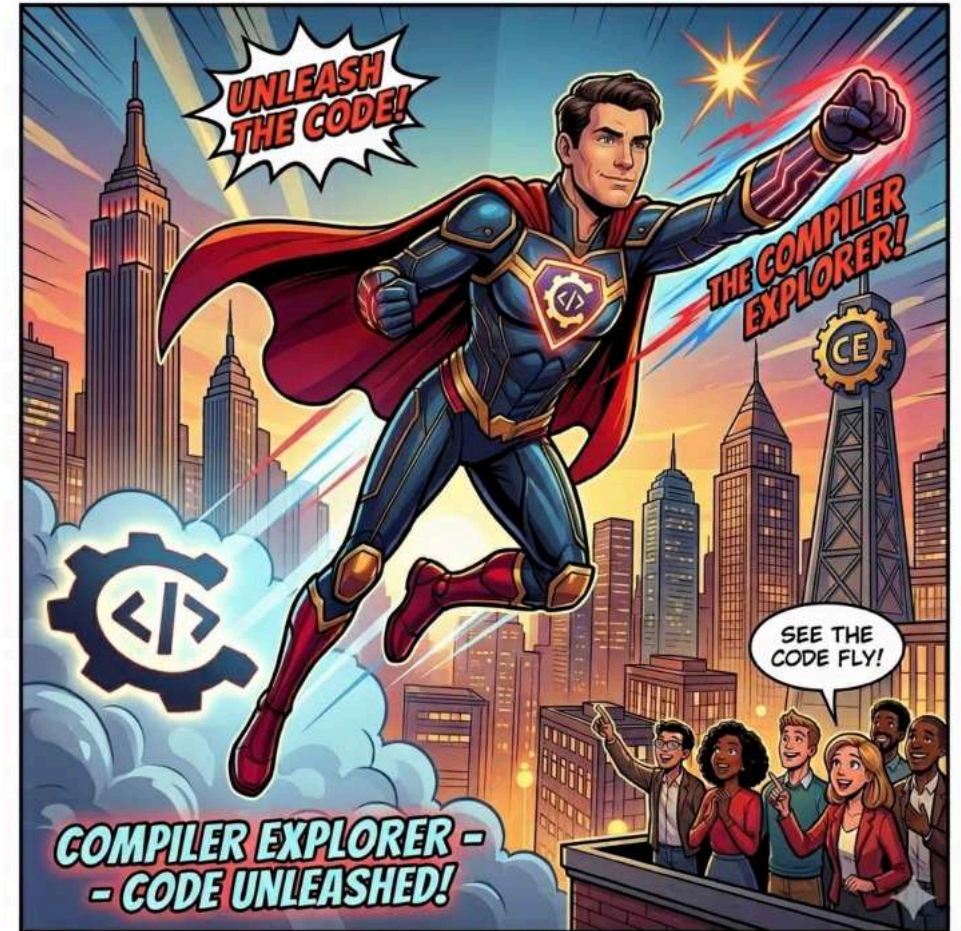
Arguably the most important C++ tool of the last decade.



Compiler Explorer

Arguably the most important C++ tool of the last decade.

- **Zero-install** — play with code before downloading anything
- **Instant sharing** — one link, no repo, no setup
- **Library support** — especially easy for header-only libraries
- **Responsive team** — reach out, they will help you add your library



Compiler Explorer: Full Lifecycle Coverage

Prototyping

- Explore ideas in seconds
- No build system needed

Compiler Explorer: Full Lifecycle Coverage

Prototyping

- Explore ideas in seconds
- No build system needed

Design Discussion

- Share a link instead of a wall of code
- "Here's what I mean" in one click

Compiler Explorer: Full Lifecycle Coverage

Prototyping

- Explore ideas in seconds
- No build system needed

Design Discussion

- Share a link instead of a wall of code
- "Here's what I mean" in one click

Bug Reports

- Reproduction link *required* in issue templates
- Eliminates the "it doesn't work on my machine" loop

Compiler Explorer: Full Lifecycle Coverage

Prototyping

- Explore ideas in seconds
- No build system needed

Design Discussion

- Share a link instead of a wall of code
- "Here's what I mean" in one click

Bug Reports

- Reproduction link *required* in issue templates
- Eliminates the "it doesn't work on my machine" loop

Documentation

- "Try it out" links in README and docs
- Interactive embeds in tutorials

Compiler Explorer: Full Lifecycle Coverage

Prototyping

- Explore ideas in seconds
- No build system needed

Design Discussion

- Share a link instead of a wall of code
- "Here's what I mean" in one click

Bug Reports

- Reproduction link *required* in issue templates
- Eliminates the "it doesn't work on my machine" loop

Zero-Overhead Proof

- Show the assembly — don't just claim it
- Let skeptics verify your abstractions cost nothing

Documentation

- "Try it out" links in README and docs
- Interactive embeds in tutorials

The Zero-Overhead Proof

Don't claim zero overhead — show the assembly.

The Zero-Overhead Proof

Don't claim zero overhead — show the assembly.

NO UNITS LIBRARY

```
double ttg_s(double d_m, double v_mps)
{
    return d_m / v_mps;
}
```

```
ttg_s(double, double):
    divsd xmm0, xmm1
    ret
```

MP-UNITS

```
quantity<s> ttg(quantity<m> d, quantity<m/s> v)
{
    return d / v;
}
```

```
ttg(quantity<m>, quantity<m/s>):
    divsd xmm0, xmm1
    ret
```

The Zero-Overhead Proof

Don't claim zero overhead — show the assembly.

NO UNITS LIBRARY

```
double ttg_s(double d_m, double v_mps)
{
    return d_m / v_mps;
}
```

```
ttg_s(double, double):
    divsd xmm0, xmm1
    ret
```

MP-UNITS

```
quantity<s> ttg(quantity<m> d, quantity<m/s> v)
{
    return d / v;
}
```

```
ttg(quantity<m>, quantity<m/s>):
    divsd xmm0, xmm1
    ret
```

Try it for yourself: <https://godbolt.org/z/dvaq3af1T>.

Interactive tutorials

The screenshot shows the mp-units website with a blue header. The navigation bar includes links for Home, Getting Started, Tutorials, User's Guide, Workshops, How-to Guides, Examples, Reference, Blog, and Release Notes. A search bar is located on the right. The main content area is titled 'Unit Conversions' and includes a sidebar with a list of tutorials. The 'Unit Conversions' tutorial is selected, showing a description, a goal, and a time estimate. Below this, the 'The .in() Method' section is displayed, featuring a code editor with C++ code for unit conversion and a terminal window showing the program's output.

mp-units HEAD

Search

mpusz/mp-units v2.5.0 1.4k 118

Home Getting Started Tutorials User's Guide Workshops How-to Guides Examples Reference Blog Release Notes

Tutorials

- Quick Start
- Working with Units
 - Unit Conversions
 - The .in() Method
 - Conversion in Expressions
 - Common Conversions
 - Function Interfaces with Automatic Conversion
 - Challenges
 - What You Learned?
- Safe and Unsafe Conversions
- Extracting Numeric Values
- Type Safety
- Affine Space

Unit Conversions

Learn to convert quantities between different units safely and explicitly.

Goal: Convert the same quantity to different units

Time: ~15 minutes

The .in() Method

To express a quantity in different units, use `.in()`:

```
1 #include <mp-units/systems/si.h>
2 #include <mp-units/systems/yard_pound.h>
3 #include <iostream>
4
5 int main()
6 {
7     using namespace mp_units;
8     using namespace mp_units::si::unit_symbols;
9     using namespace mp_units::yard_pound::unit_symbols;
10
11     // Distance in meters
12     quantity distance = 5.0 * km;
13
14     std::cout << "Original: " << distance << "\n";
15     std::cout << "In meters: " << distance.in(m) << "\n";
16     std::cout << "In miles: " << distance.in(mi) << "\n";
17     std::cout << "In feet: " << distance.in(ft) << "\n";
18
19     // Speed conversion
```

Program returned: 0

Program stdout

```
Original: 5 km
In meters: 5000 m
In miles: 3.10686 mi
In feet: 16404.2 ft

Speed: 100 km/h
```

x86-64 clang 21.1.0 118ms

Making It Work for Your Library

GETTING ADDED TO GODBOLT

- 1 Open an issue/PR on [compiler-explorer/compiler-explorer](#)
- 2 Header-only → trivially easy
- 3 The team is **friendly and responsive**

Making It Work for Your Library

GETTING ADDED TO GODBOLT

- 1 Open an issue/PR on [compiler-explorer/compiler-explorer](#)
- 2 Header-only → trivially easy
- 3 The team is **friendly and responsive**

BEST PRACTICES

- Use **shortlinks** ([godbolt.org/z/abc123](#)) — keep docs clean
- Add "▶ **Try it on Compiler Explorer**" links to every README example
- Require a **Godbolt reproduction link** in your bug report template
- Use **interactive embeds** for doc tutorials

Making It Work for Your Library

GETTING ADDED TO GODBOLT

- 1 Open an issue/PR on [compiler-explorer/compiler-explorer](#)
- 2 Header-only → trivially easy
- 3 The team is **friendly and responsive**

BEST PRACTICES

- Use **shortlinks** ([godbolt.org/z/abc123](#)) — keep docs clean
- Add "► **Try it on Compiler Explorer**" links to every README example
- Require a **Godbolt reproduction link** in your bug report template
- Use **interactive embeds** for doc tutorials

Please do provide Release tags for your library → stability guarantee.

TIP 6

ONE-CLICK DEV ENVIRONMENTS

User's Scenario: "I want to fix a typo"



One-Click Dev Environments

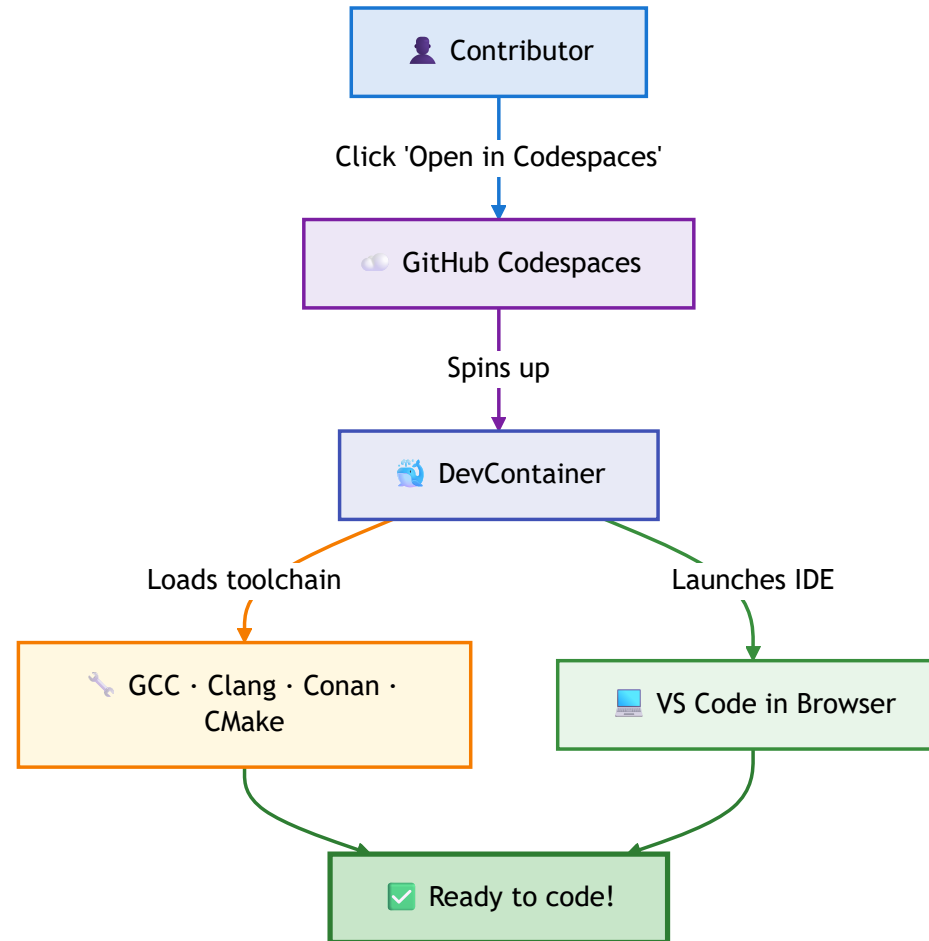
Instead of describing the development environment provide one as a container.

One-Click Dev Environments

Instead of describing the development environment provide one as a container.

Toolchain, Config files, Dependencies pre-installed.

GitHub Codespaces: Easy Contributing



devcontainer.json: Toolchain as Code

```
{
  "image":
    "trainiteu/mp-units:latest",
  "postCreateCommand":
    ".devcontainer/post-create.sh",
  "customizations": {
    "vscode": {
      "extensions": [
        "ms-vscode.cpptools",
        "ms-vscode.cmake-tools",
        "..."
      ]
    }
  }
}
```

WHAT'S PRE-INSTALLED

-  **Compilers:** GCC 12–15 · Clang 16–22
-  **Build:** CMake · Ninja · Conan 2.x
-  **Analysis:** clang-format · clang-tidy · IWYU
-  **Docs:** MkDocs + Material theme
-  **Profiles:** All Conan configs pre-set
-  **Cache:** Dependencies pre-compiled

devcontainer.json: Toolchain as Code

```
{
  "image":
    "trainiteu/mp-units:latest",
  "postCreateCommand":
    ".devcontainer/post-create.sh",
  "customizations": {
    "vscode": {
      "extensions": [
        "ms-vscode.cpptools",
        "ms-vscode.cmake-tools",
        "..."
      ]
    }
  }
}
```

WHAT'S PRE-INSTALLED

-  **Compilers:** GCC 12–15 · Clang 16–22
-  **Build:** CMake · Ninja · Conan 2.x
-  **Analysis:** clang-format · clang-tidy · IWYU
-  **Docs:** MkDocs + Material theme
-  **Profiles:** All Conan configs pre-set
-  **Cache:** Dependencies pre-compiled

Remember to test this in CI as well.

Not Just for External Contributors

TEAM CONSISTENCY

- **Identical** environment for every developer
- No "works on my machine" syndrome
- New team member → productive on day 1
- Container = CI image → **local == CI**

Not Just for External Contributors

TEAM CONSISTENCY

- **Identical** environment for every developer
- No "works on my machine" syndrome
- New team member → productive on day 1
- Container = CI image → **local == CI**

RUN LOCALLY TOO

- Docker + VS Code Dev Containers extension
→ **"Reopen in Container"**
- Full offline development
 - No cloud usage limits
 - Better performance on local hardware

Not Just for External Contributors

TEAM CONSISTENCY

- **Identical** environment for every developer
- No "works on my machine" syndrome
- New team member → productive on day 1
- Container = CI image → **local == CI**

ISOLATED PR TESTING

```
# Fresh Codespace per PR branch
conan build . -pr clang21 \
  -c user.mp-units.build:all=True -b missing
```

RUN LOCALLY TOO

- Docker + VS Code Dev Containers extension
→ **"Reopen in Container"**
- Full offline development
 - No cloud usage limits
 - Better performance on local hardware

Not Just for External Contributors

TEAM CONSISTENCY

- **Identical** environment for every developer
- No "works on my machine" syndrome
- New team member → productive on day 1
- Container = CI image → **local == CI**

ISOLATED PR TESTING

```
# Fresh Codespace per PR branch
conan build . -pr clang21 \
  -c user.mp-units.build:all=True -b missing
```

RUN LOCALLY TOO

- Docker + VS Code Dev Containers extension
→ **"Reopen in Container"**
- Full offline development
 - No cloud usage limits
 - Better performance on local hardware

FULL MATRIX TESTING

```
# All compilers – similar to CI
.devcontainer/check_all.sh create
```

GCC 12–15 · Clang 16–22 · C++20/23/26

Trivial local reproduction

Clang-20 (x86-64) C++23 std::format modules=yes import_std=yes no_crtp=yes none Release summary ...

✗ Freestanding Build Failed: Clang-20 (x86-64) C++23 std::format modules=yes import_std=yes no_crtp=yes none Release

Local repro (in the provided '.devcontainer' or GitHub CodeSpaces environment):

```
conan build . -b missing -s compiler=clang -s compiler.version=20 -s compiler.cppstd=23 -s build_type=Rel
```

Job summary generated at run-time

TIP 7

SEAMLESS INTEGRATION

Don't be asocial!



David Sankel: Big Projects, and CMake, and Git, Oh My!

Don't be asocial!

A library is asocial when it lacks proper means to consume it.

Don't be asocial!

A library is asocial when it lacks proper means to consume it.

✗ ASOCIAL LIBRARY

- Custom **build.sh** script required
- No CMake exported targets
- No package manager recipe
- Obscure or home-grown dependencies
- Hardcoded compiler flags

Don't be asocial!

A library is asocial when it lacks proper means to consume it.

❌ ASOCIAL LIBRARY

- Custom **build.sh** script required
- No CMake exported targets
- No package manager recipe
- Obscure or home-grown dependencies
- Hardcoded compiler flags

✅ SOCIAL LIBRARY

- Well-known build tools and dependencies
- **find_package()** + namespaced targets
- On [Conan Center](#) and/or [vcpkg](#)
- Out-of-source, CMake Presets support
- User's flags are respected

Don't be asocial!

A library is asocial when it lacks proper means to consume it.

❌ ASOCIAL LIBRARY

- Custom **build.sh** script required
- No CMake exported targets
- No package manager recipe
- Obscure or home-grown dependencies
- Hardcoded compiler flags

✅ SOCIAL LIBRARY

- Well-known build tools and dependencies
- **find_package()** + namespaced targets
- On [Conan Center](#) and/or [vcpkg](#)
- Out-of-source, CMake Presets support
- User's flags are respected

Principle: Provide the integration you wish you had as a user.

Package Management: Conan & vcpkg

conanfile.txt

```
[requires]
mp-units/2.5.0

[options]
mp-units*:std_format=True
mp-units*:contracts=gsl-lite

[generators]
CMakeToolchain
CMakeDeps
```

```
conan install . -pr gcc15 -b missing
cmake --preset conan-default
cmake --build --preset conan-release
```

Package Management: Conan & vcpkg

conanfile.txt

```
[requires]
mp-units/2.5.0

[options]
mp-units*:std_format=True
mp-units*:contracts=gsl-lite

[generators]
CMakeToolchain
CMakeDeps
```

```
conan install . -pr gcc15 -b missing
cmake --preset conan-default
cmake --build --preset conan-release
```

FOR USERS

- Zero manual dependency management
- *Feature flags* become Conan options
- *Reproducible builds* across teams

Meet Users Where They Are

Not everyone uses a package manager — and that's okay.

Meet Users Where They Are

Not everyone uses a package manager — and that's okay.

LEGACY & CORPORATE ENVIRONMENTS

-  **Strict dependency policies** — no external package managers allowed
-  **Air-gapped networks** — no internet access at build time
-  **Existing monorepos** — vendoring is the standard

Meet Users Where They Are

Not everyone uses a package manager — and that's okay.

LEGACY & CORPORATE ENVIRONMENTS

-  **Strict dependency policies** — no external package managers allowed
-  **Air-gapped networks** — no internet access at build time
-  **Existing monorepos** — vendoring is the standard

NICHE & EXPERIMENTAL BUILDS

-  **CPM / FetchContent** — CMake-native, no extra tools
-  **Bazel, Meson, build2** — not everyone uses CMake
-  **add_subdirectory()** — simple vendoring, zero overhead

Supported Consumption Paths

- 🏆 **Conan Center** — official releases
- 🔗 **vcpkg** — ships with Visual Studio
- 🚀 **CPM / FetchContent** — no package manager
- 📁 **add_subdirectory()** — vendoring
- 🛠️ **Install + find_package()** — system-wide

Supported Consumption Paths

- 🏆 **Conan Center** — official releases
- 🔗 **vcpkg** — ships with Visual Studio
- 🚀 **CPM / FetchContent** — no package manager
- 📁 **add_subdirectory()** — vendoring
- 🛠️ **Install + find_package()** — system-wide

The Principle: The more paths you support, the fewer reasons to say no.

Modern CMake: Targets, Not Variables

CONSUMER'S CMakeLists.txt

```
find_package(mp-units REQUIRED)

target_link_libraries(my_app
    PRIVATE
        mp-units::mp-units
)
```


Modern CMake: Targets, Not Variables

CONSUMER'S CMakeLists.txt

```
find_package(mp-units REQUIRED)

target_link_libraries(my_app
    PRIVATE
    mp-units::mp-units
)
```

WHAT TARGETS GIVE YOU?

- ✓ Include paths — automatic
- ✓ Compile flags — propagated
- ✓ Transitive dependencies — resolved
- ✓ **PRIVATE/PUBLIC/INTERFACE** — respects user's graph
- ✓ Install rules — **find_package()** works after **cmake --install**

Modern CMake: Targets, Not Variables

CONSUMER'S CMakeLists.txt

```
find_package(mp-units REQUIRED)

target_link_libraries(my_app
    PRIVATE
    mp-units::mp-units
)
```

WHAT TARGETS GIVE YOU?

- ✓ Include paths — automatic
- ✓ Compile flags — propagated
- ✓ Transitive dependencies — resolved
- ✓ **PRIVATE/PUBLIC/INTERFACE** — respects user's graph
- ✓ Install rules — **find_package()** works after **cmake --install**

Avoid: `include_directories()`, playing with `CMAKE_CXX_FLAGS`, changing ABI related flags, or setting any compilation warning flags.

Wide Compatibility: The C++ Reality

Feature support varies wildly across the compiler matrix.

Wide Compatibility: The C++ Reality

Feature support varies wildly across the compiler matrix.

FEATURE	GCC	CLANG	MSVC
Minimum (C++20)	12+	16+, 19	194+
<code>std::format</code>	13+	17+	194+
C++ modules	—	17+	—
<code>import std;</code>	—	18+	—
<code>explicit this</code> (no CRTP)	14+	18+	194+

Wide Compatibility: The C++ Reality

Feature support varies wildly across the compiler matrix.

FEATURE	GCC	CLANG	MSVC
Minimum (C++20)	12+	16+, 19	194+
<code>std::format</code>	13+	17+	194+
C++ modules	—	17+	—
<code>import std;</code>	—	18+	—
<code>explicit this</code> (no CRTP)	14+	18+	194+

Your users are spread across every cell of this table. Meet them all!

Wide Compatibility: Choose Your Style

CONAN CONFIGURATION OPTIONS

OPTION	VALUES
<code>cxx_modules</code>	<code>True/False</code>
<code>import_std</code>	<code>True/False</code>
<code>std_format</code>	<code>True/False</code>
<code>no_crtp</code>	<code>True/False</code>
<code>contracts</code>	<code>gsl-lite/ms-gsl/none</code>
<code>freestanding</code>	<code>True/False</code>

Wide Compatibility: Choose Your Style

CONAN CONFIGURATION OPTIONS

OPTION	VALUES
cxx_modules	True/False
import_std	True/False
std_format	True/False
no_crtp	True/False
contracts	gsl-lite/ms-gsl/none
freestanding	True/False

STRATEGY

- Targeting latest compilers? → **terse modern style**
- Need portability? → **compatibility macros**
- Start portable, refactor later

Wide Compatibility: Choose Your Style

CONAN CONFIGURATION OPTIONS

OPTION	VALUES
cxx_modules	True/False
import_std	True/False
std_format	True/False
no_crtp	True/False
contracts	gsl-lite/ms-gsl/none
freestanding	True/False

STRATEGY

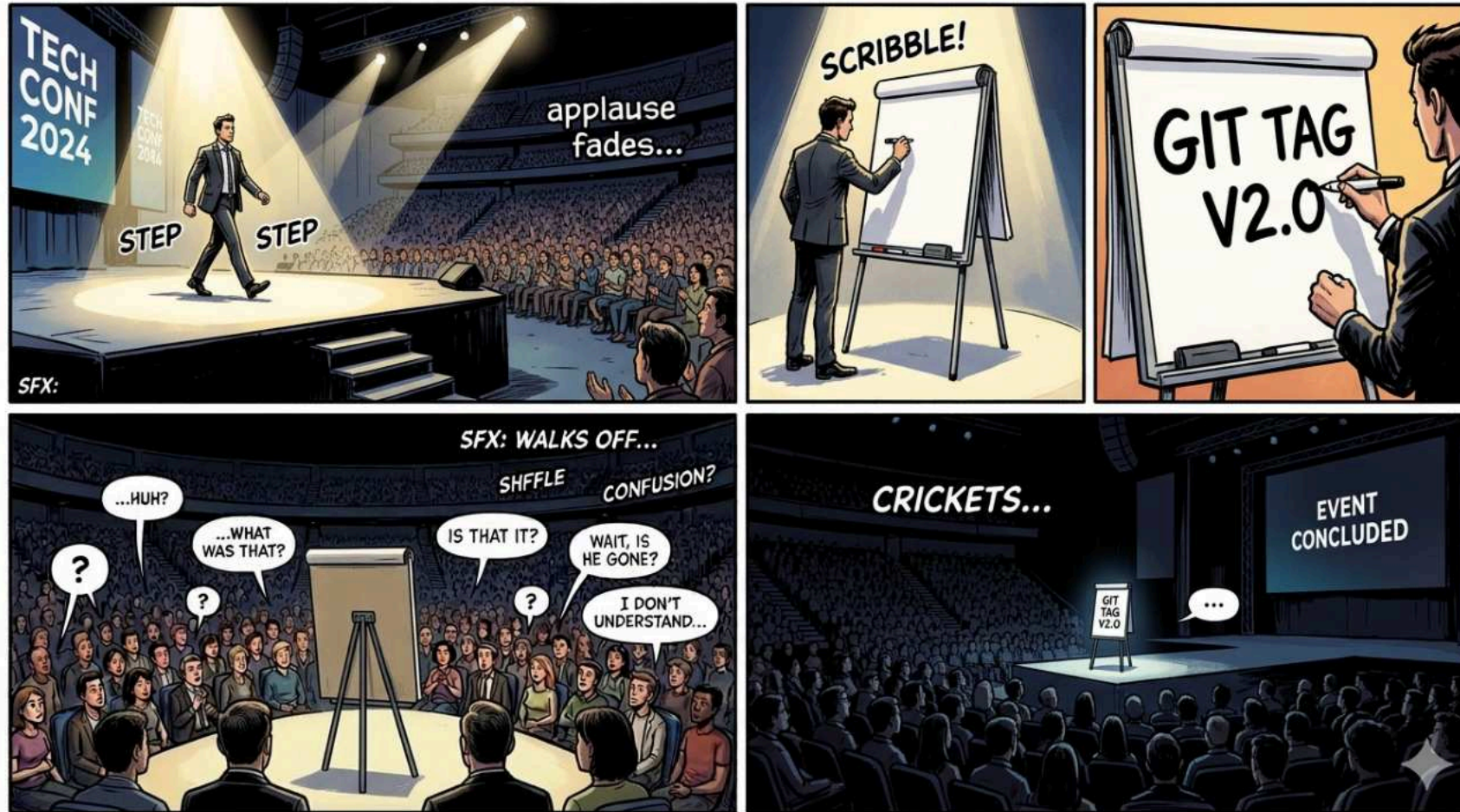
- Targeting latest compilers? → **terse modern style**
- Need portability? → **compatibility macros**
- Start portable, refactor later

All combinations tested in the CI matrix!

TIP 8

THE RELEASE BLOG

A git tag is not user communication



A git tag is not user communication

Releases are marketing events.

A git tag is not user communication

Releases are marketing events.

A tag on GitHub is not a release announcement.

Prose-Based Release Notes

- **The "Why":** Why did this version happen?
- **Highlights:** What are the 3 big things?
- **Breaking Changes:** How do I fix my code?

The Blog Post

 **mp-units** HEAD ▾

  Search

 mpusz/mp-units
v2.5.0 1.4k 118

HomeGetting StartedTutorialsUser's GuideWorkshopsHow-to GuidesExamplesReferenceBlogRelease Notes

[← Back to index](#)

 **Mateusz Pusz**
Creator

Metadata

 November 5, 2024

 in Releases

 8 min read

Table of contents

ISQ quantities cleanup

Units equality

Portable text output

char_traits removed from fixed_string

Improved units' text output

Unicode identifiers

Convertibility with QuantityLike and QuantityPointLike entities

Symbolic constants implementation should be implementation-defined

mp-units 2.4.0 Released

A new product version can be obtained from [GitHub](#) and [Conan](#).

This release was unexpected. We planned a significant new feature to happen next, but while preparing for it and writing API Reference documentation, we made so many vital fixes and improvements that we decided they deserve a dedicated release first.

This post describes the most significant improvements while a much longer list of the changes introduced by the new version can be found in our [Release Notes](#).

ISQ quantities cleanup

Initially, we kept quantities defined in "IEC 80000-13: Information science and technology" in a standalone `iec80000` namespace, which was renamed to `iec` in the previous release. It turned out that this was incorrect. Those quantities are also a part of the ISQ. This is why, in this release, we moved all of them to the `isq` namespace (✖ **breaking change** ✖).

From now on, `iec` namespace does not provide any quantities and serves purely as a system of units definition. It contains binary prefixes (based on the powers of two) and some units introduced by IEC (e.g., `var`, `erlang`, `bit`, or `baud`).

 **Note**

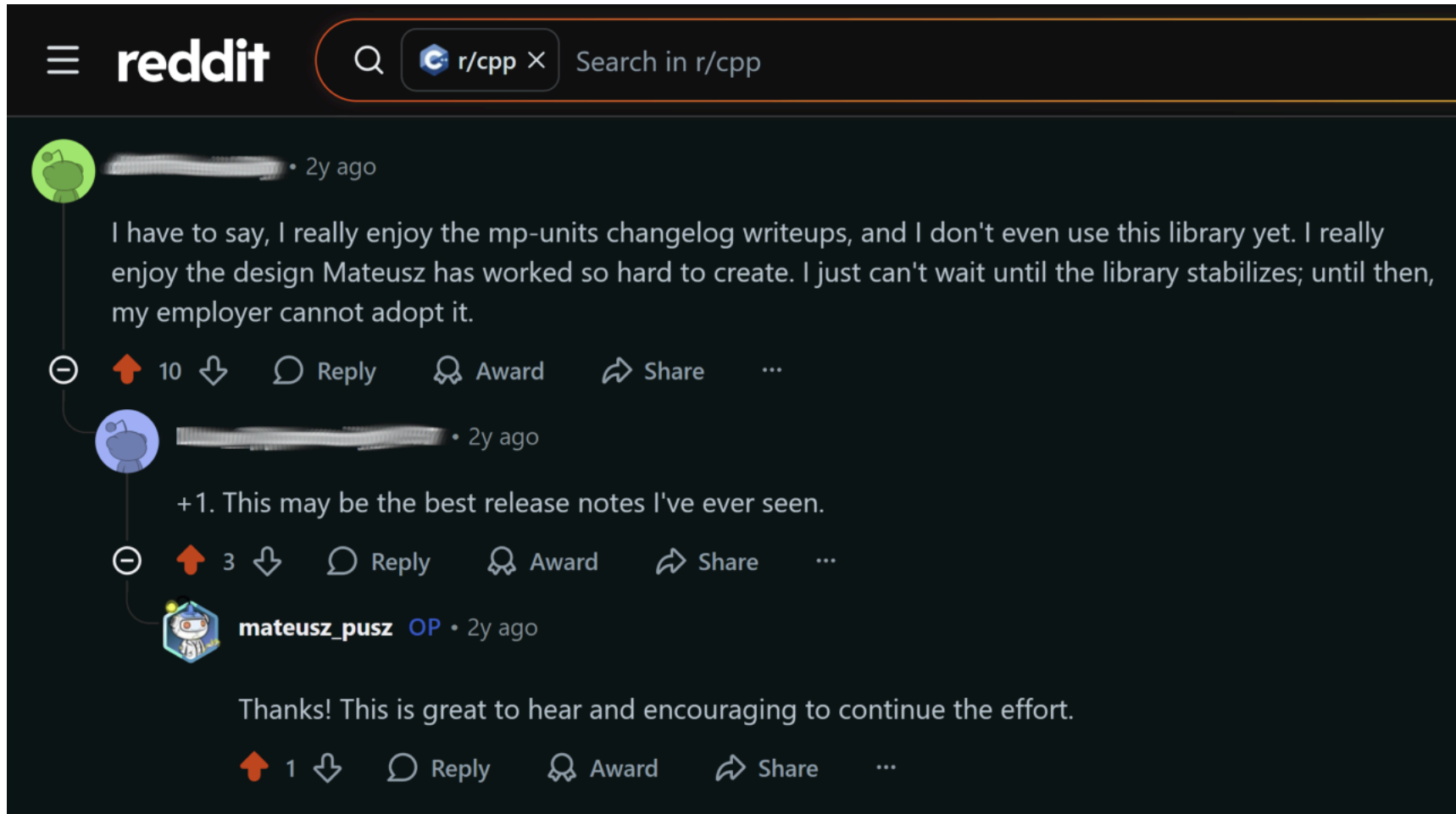
The quantities in `iec` namespace are now deprecated and will be removed in future releases.

Also, it turns out that the latest ISO 80000-3 revision makes a small cleanup to the `phase_speed` and `group_speed` quantities. Those were always defined as scalar quantities but also had alternative names `phase_velocity` and `group_velocity`. This is misleading as *velocity* is typically considered a vector quantity. It is why those `XXX_velocity` aliases were removed from the ISO

Distribution

- [LinkedIn](#) for the professional network
- [Reddit](#) ([r/cpp](#)) for the technical crowd
- [Twitter/X](#) for the "breaking news"

Distribution



TIP 9

A CULTURE OF APPRECIATION

Why Do People Contribute to Open Source?

The Reality: Open source runs on human motivation.

Why Do People Contribute to Open Source?

The Reality: Open source runs on human motivation.

TWO PRIMARY DRIVERS

- 1 **Curiosity:** "I want to learn bleeding-edge C++"
- 2 **Career:** "I need portfolio projects for job applications"

Why Do People Contribute to Open Source?

The Reality: Open source runs on human motivation.

TWO PRIMARY DRIVERS

- 1 **Curiosity:** "I want to learn bleeding-edge C++"
- 2 **Career:** "I need portfolio projects for job applications"

Your Job: Make both rewarding.

Building Contributor CVs (Win-Win)

The Problem: Junior developers need proof of impact.

Building Contributor CVs (Win-Win)

The Problem: Junior developers need proof of impact.

1 **CONTRIBUTORS.md:** Maintain a contributors page with

- Names hyperlinked to their GitHub profiles
- Specific contributions listed
- Make it easy to refer the contribution in their resume

Building Contributor CVs (Win-Win)

The Problem: Junior developers need proof of impact.

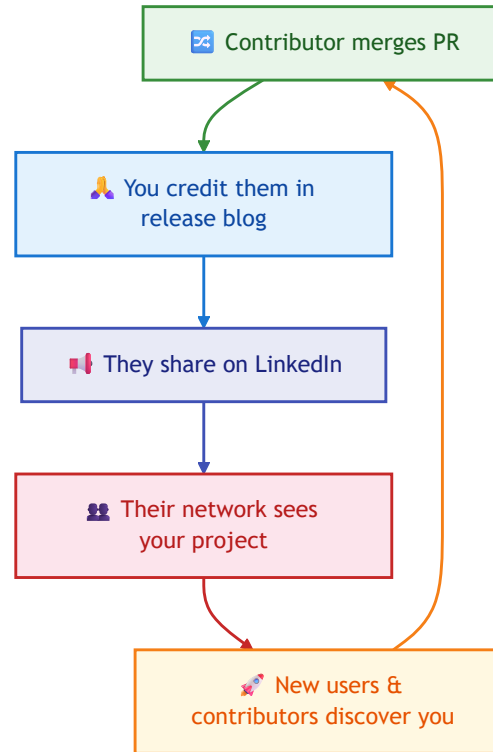
- 1 **CONTRIBUTORS.md:** Maintain a contributors page with
 - Names hyperlinked to their GitHub profiles
 - Specific contributions listed
 - Make it easy to refer the contribution in their resume
- 2 **Release Blog Credits:** "Special thanks to **@jdoe** for the new SI conversions!"

Building Contributor CVs (Win-Win)

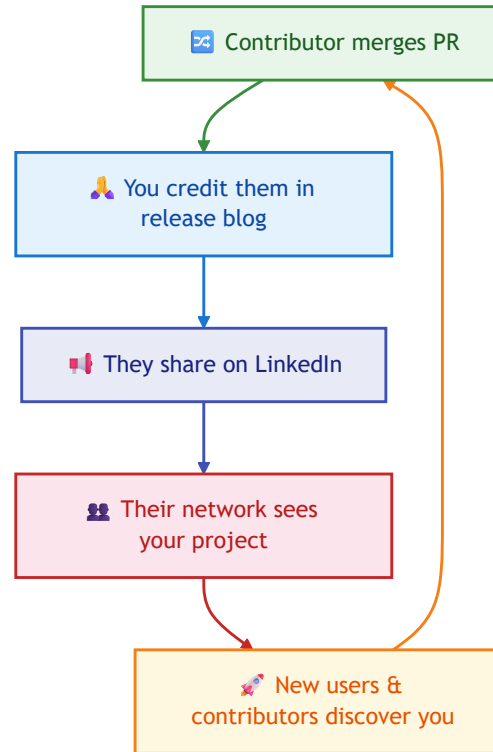
The Problem: Junior developers need proof of impact.

- 1 **CONTRIBUTORS.md:** Maintain a contributors page with
 - Names hyperlinked to their GitHub profiles
 - Specific contributions listed
 - Make it easy to refer the contribution in their resume
- 2 **Release Blog Credits:** "Special thanks to **@jdoe** for the new SI conversions!"
- 3 **Testimonials:** Offer to write LinkedIn recommendations for significant contributors

The Multiplier Effect



The Multiplier Effect



People share their contributions → Free marketing for your project.

Recognition Strategies

EASY WINS

-  Always say **"Thank you!"** (even for bad bug reports)
-  *Mention contributors* by name in release notes
-  Use GitHub's *"Co-authored-by"* for pair programming
-  Create a *"Hall of Fame"* section in docs
-  Share contributor spotlights on *social media*

Recognition Strategies

EASY WINS

-  Always say **"Thank you!"** (even for bad bug reports)
-  *Mention contributors* by name in release notes
-  Use GitHub's *"Co-authored-by"* for pair programming
-  Create a *"Hall of Fame"* section in docs
-  Share contributor spotlights on *social media*

ADVANCED

-  *Help* contributors write blog posts about their work
-  *Invite* them to co-present at conferences
-  *Provide* reference letters for job applications

Managing the AI Influx

THE CHALLENGE

- LLM-generated PRs are increasing



Managing the AI Influx

THE CHALLENGE

- LLM-generated PRs are increasing

THE DANGER

- *Low-effort, high-volume* contributions
- AI *refactors entire designs* without understanding rationale
- Generates *"average"* code (learns from poor examples)
- Creates *massive diffs* that are impossible to review



Managing the AI Influx

THE CHALLENGE

- LLM-generated PRs are increasing

THE DANGER

- *Low-effort, high-volume* contributions
- AI *refactors entire designs* without understanding rationale
- Generates *"average"* code (learns from poor examples)
- Creates *massive diffs* that are impossible to review

You need a clear AI policy



AI Contribution Policy

AI-Generated Contributions

We welcome the use of LLMs (Large Language Models) to help you write better prose, generate unit test cases, or brainstorm solutions. However, to maintain the high technical standards of **mp-units**, we ask you to follow these rules:

1. **Rationale First:** Never submit a "blind refactor." If an AI suggests a different design, you must be able to explain the rationale behind it.
2. **Review Your Diff:** Ensure the AI hasn't rewritten entire files unnecessarily. Large, monolithic diffs for small changes will be rejected.
3. **Verify Quality:** AI often suggests "average" code. Since this is a bleeding-edge library using C++20/23, ensure the AI hasn't introduced legacy patterns or inefficient templates.
4. **Transparency:** If a significant portion of your PR was generated by AI, please disclose it. This helps reviewers focus on correctness rather than style.

The "Primary Author" Principle

Protect Design Integrity.



The "Primary Author" Principle

Protect Design Integrity.

❌ **Don't:** Replace a class design just because AI suggested it

✅ **Do:** Ask the primary author about their rationale first



The "Primary Author" Principle

REMEMBER

- The original design often had **specific reasons**
 - *Compile-time performance* considerations
 - *Edge cases* the AI doesn't see
 - *Consistency* with the broader architecture



The "Primary Author" Principle

REMEMBER

- The original design often had **specific reasons**
 - *Compile-time performance* considerations
 - *Edge cases* the AI doesn't see
 - *Consistency* with the broader architecture



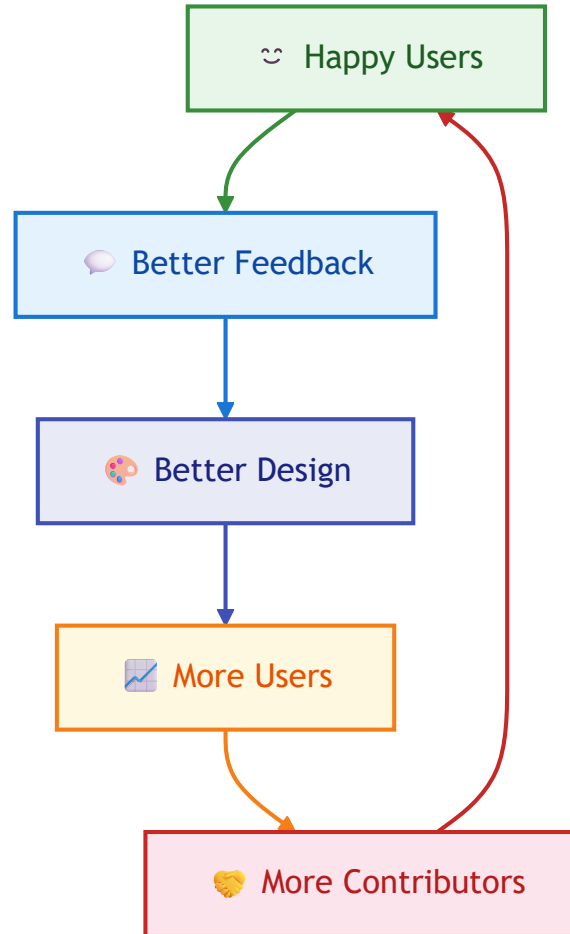
AI often learns from poor code - your library is likely better.

Building a Welcoming Culture

THE FOUNDATION

- **Be respectful** to all users and contributors
- Use "We" not "You" in error messages
 - e.g., "We couldn't find" vs "You provided wrong"
- **Empathy over ego** in issue responses
- Follow GitHub's [Community Standards](#)

The Feedback Loop



TIP 10

PROACTIVE COMMUNITY BUILDING

Proactive Community Building

Passive maintenance leads to project death. You must be the "Chief Evangelist".

Proactive Community Building

Passive maintenance leads to project death. You must be the "Chief Evangelist".

If you don't talk about your project, who will?

Scaling the Niche

Niche projects (like **mp-units**) need education.

Scaling the Niche

Niche projects (like **mp-units**) need education.

Write about the domain (physics, math), not just the code.

Seeking Production Feedback

- Create an issue: "I'm looking for production users."
- Be prepared for *complaints rather than praise*
- Listen to the *"boring" problems*
 - compilation time
 - header bloat
 - ...

Seeking Production Feedback

- Create an issue: "I'm looking for production users."
- Be prepared for *complaints rather than praise*
- Listen to the *"boring" problems*
 - compilation time
 - header bloat
 - ...

Always thank for feedback! Learn and improve from it.

Talk to the Community

- 1 Conference talks (like this one!)
- 2 Local meetups
- 3 Discord / Slack participation

SUMMARY

The Professional Product Checklist

- **Discovery:** Unique naming, great README
- **Trust:** Green master, matrix CI
- **Communication:** Issues, Discussions, empathy
- **Understanding:** Diátaxis documentation
- **Testing:** Compiler Explorer links
- **Contribution:** GitHub Codespaces
- **Integration:** CMake + Conan/Vcpkg
- **Announcement:** Release blogs, not just tags
- **Recognition:** Build contributor CVs, AI policy
- **Growth:** Proactive evangelism

Case Study: mp-units Journey

From Personal Repo to Community Project.

Case Study: mp-units Journey

From Personal Repo to Community Project.

-  **Growth:** Personal hobby → ISO standardization candidate
-  **Stars:** Track against competitors
-  **Documentation:** Single README → Comprehensive Diátaxis site
-  **Contributors:** Solo → Growing community with clear contribution path
-  **Production Use:** Actively seeking feedback from real deployments

Case Study: mp-units Journey

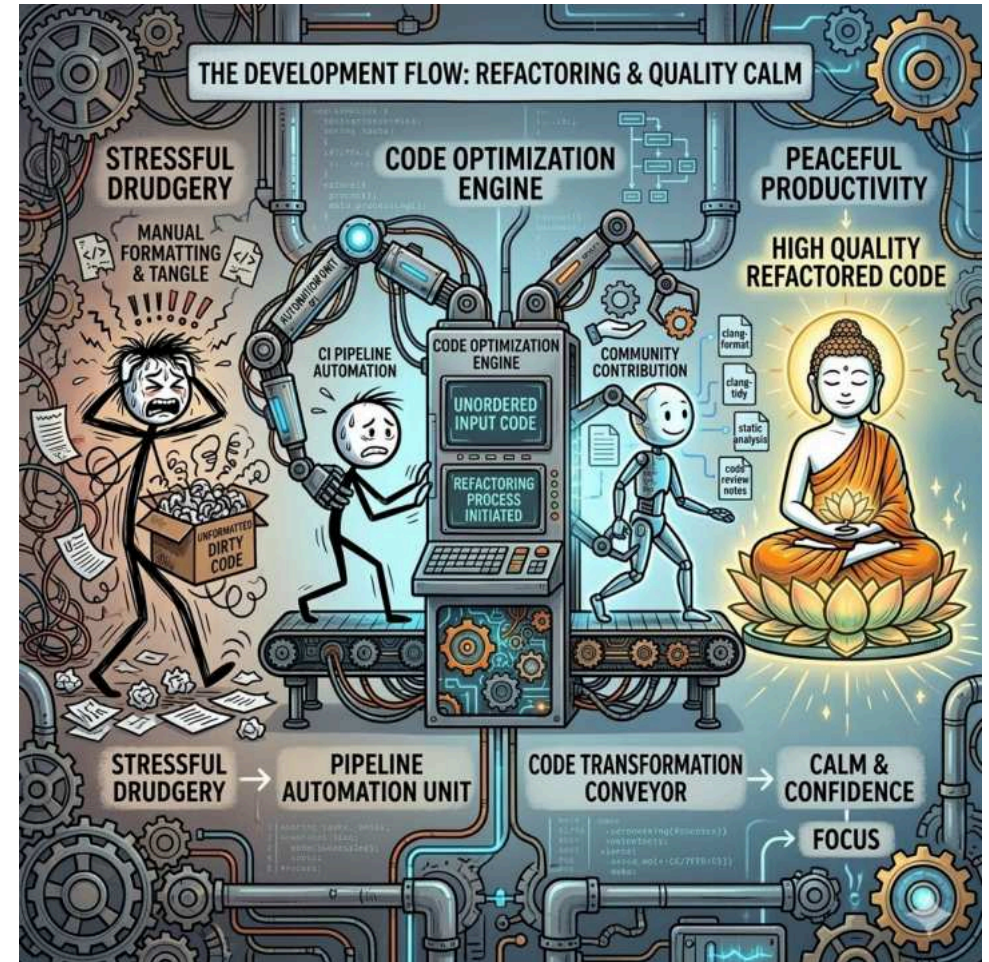
From Personal Repo to Community Project.

-  **Growth:** Personal hobby → ISO standardization candidate
-  **Stars:** Track against competitors
-  **Documentation:** Single README → Comprehensive Diátaxis site
-  **Contributors:** Solo → Growing community with clear contribution path
-  **Production Use:** Actively seeking feedback from real deployments

Key Lesson: Each of the 10 tips contributed to compound growth.

Avoiding Maintainer Burnout

You Can't Do It All - And You Shouldn't.

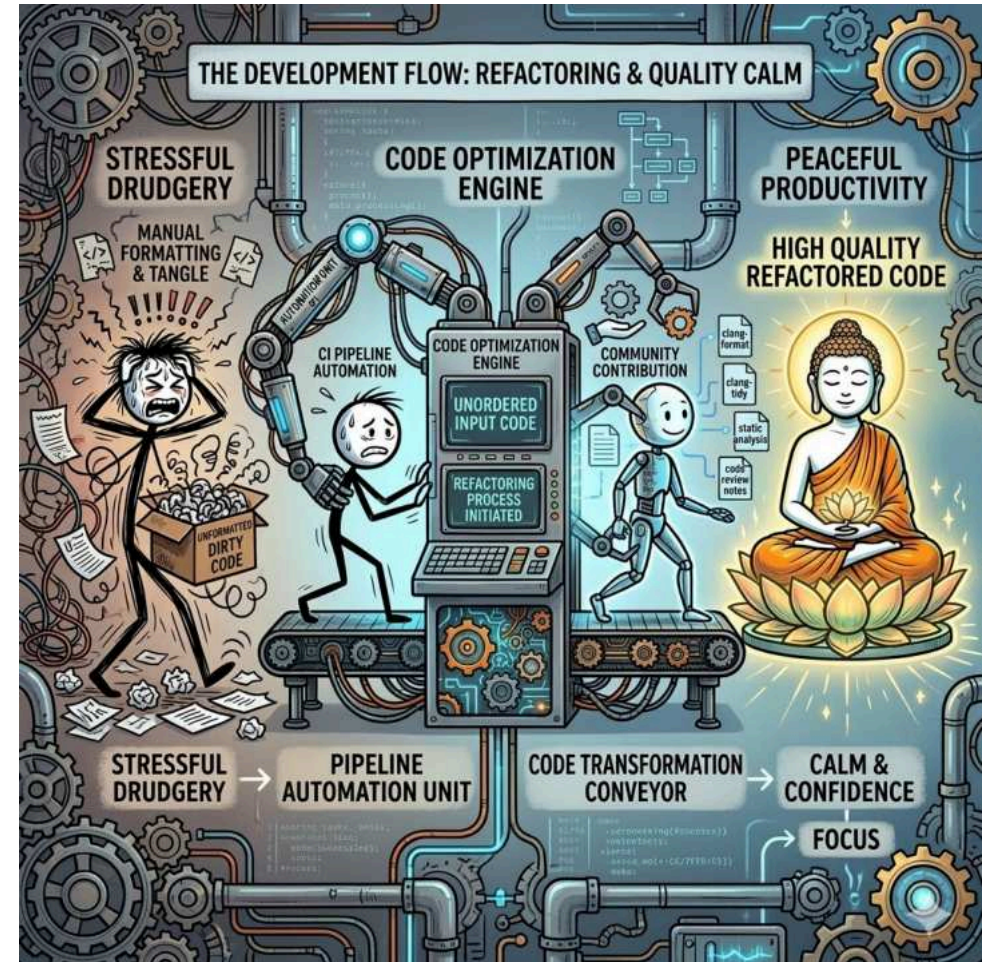


Avoiding Maintainer Burnout

You Can't Do It All - And You Shouldn't.

AUTOMATE THE BORING

- ✓ *CI* handles builds and tests
- ✓ *Clang-format* handles code style debates
- ✓ *Bots* handle stale issues
- ✓ *Release scripts or AI* generate changelogs

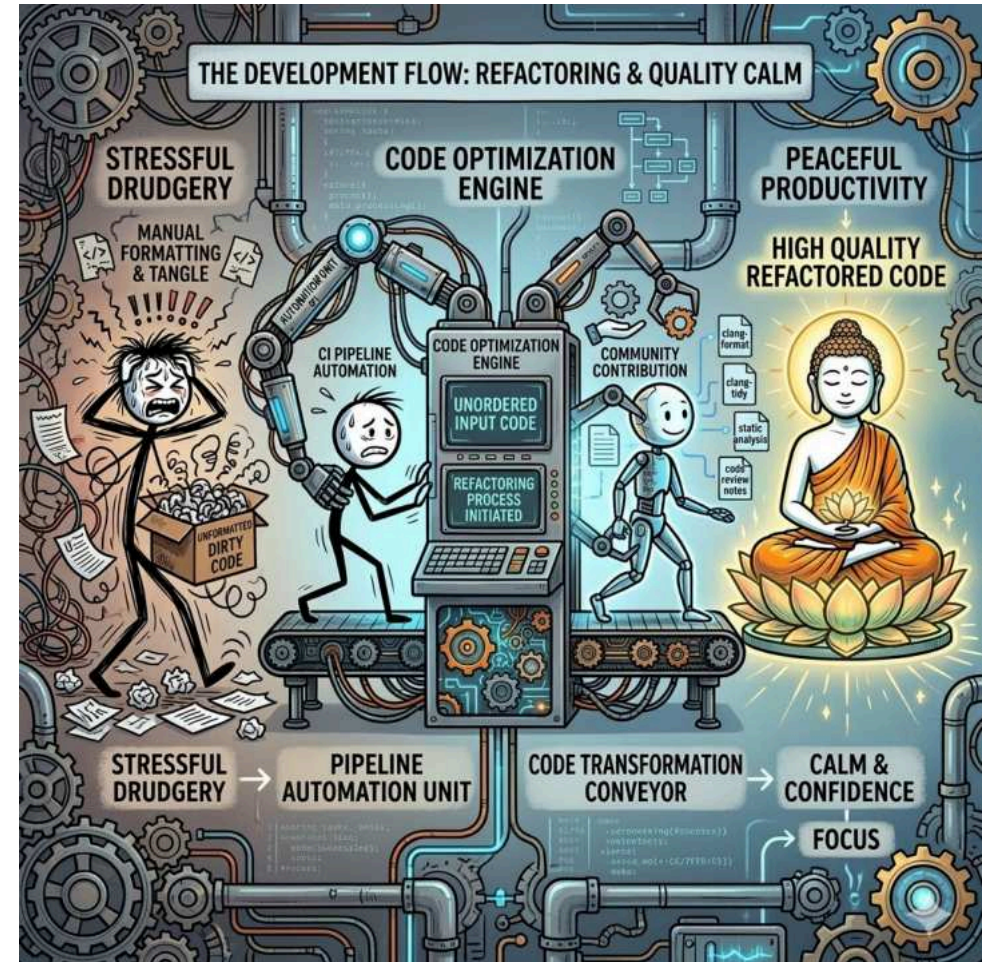


Avoiding Maintainer Burnout

You Can't Do It All - And You Shouldn't.

EMPOWER OTHERS

- ✓ Give *triage permissions* to active contributors
- ✓ Document your *decision-making process*
- ✓ It's okay to say "Not for this release"



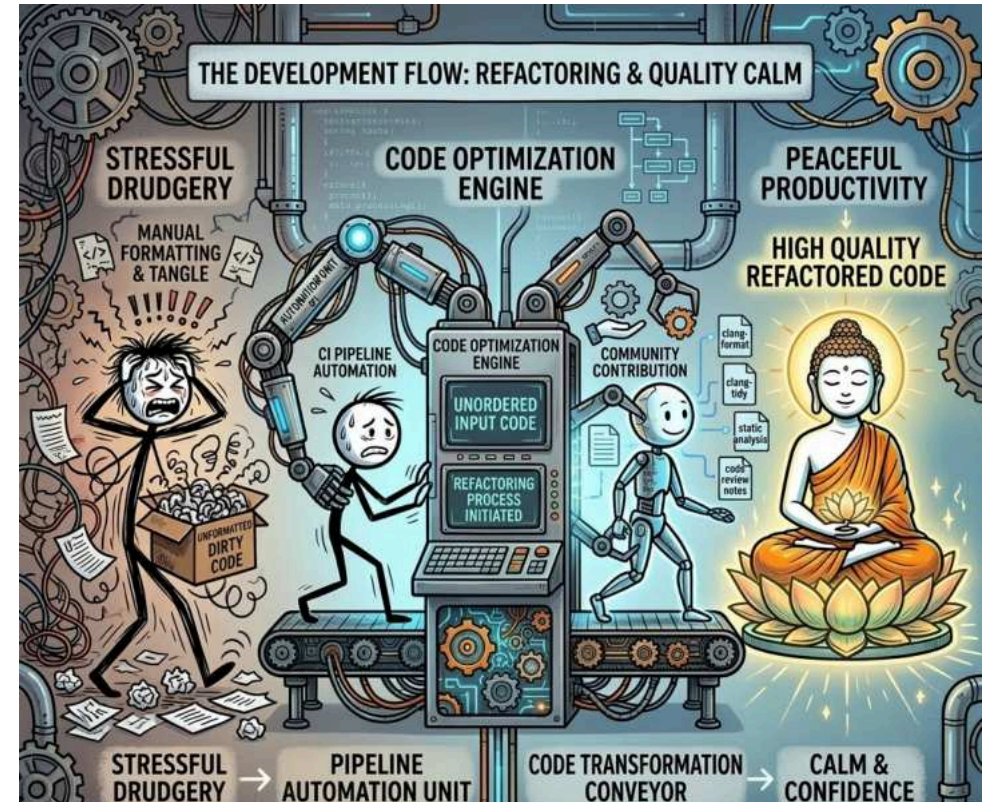
Avoiding Maintainer Burnout

You Can't Do It All - And You Shouldn't.

EMPOWER OTHERS

- ✓ Give *triage permissions* to active contributors
- ✓ Document your *decision-making process*
- ✓ It's okay to say "Not for this release"

Protect your time and energy.



Advice for Contributors (Juniors)

"Which project should I contribute to learn Modern C++?"

Advice for Contributors (Juniors)

"Which project should I contribute to learn Modern C++?"

LOOK FOR THESE GREEN FLAGS

-  Has **CONTRIBUTING.md** and good-first-issues
-  Maintainers respond kindly to questions
-  Has GitHub Codespaces (one-click setup)
-  Active CI with clear error messages
-  Documentation explains the "why"

Advice for Contributors (Juniors)

"Which project should I contribute to learn Modern C++?"

LOOK FOR THESE GREEN FLAGS

-  Has **CONTRIBUTING.md** and good-first-issues
-  Maintainers respond kindly to questions
-  Has GitHub Codespaces (one-click setup)
-  Active CI with clear error messages
-  Documentation explains the "why"

Don't be afraid of bleeding-edge repos if they're welcoming!

Advice for Maintainers (Authors)

Be the maintainer you wish you had!

Advice for Maintainers (Authors)

Be the maintainer you wish you had!

WHEN REVIEWING PRS

- 🙏 Always thank the contributor first
- 💡 Explain your design decisions, don't just reject
- 🎯 Guide them to success, don't just point out failures
- 📖 When asking for changes, link to examples

Advice for Maintainers (Authors)

Be the maintainer you wish you had!

WHEN REVIEWING PRS

- 🙏 Always thank the contributor first
- 💡 Explain your design decisions, don't just reject
- 🎯 Guide them to success, don't just point out failures
- 📖 When asking for changes, link to examples

Remember: Every "no" without explanation discourages future contributions.

From Fortress to Forum



BEFORE: THE FORTRESS

- Hard to discover
- Brittle builds
- Hostile to questions
- Doxygen dictionary
- "Figure it out yourself"

From Fortress to Forum



BEFORE: THE FORTRESS

- Hard to discover
- Brittle builds
- Hostile to questions
- Doxygen dictionary
- "Figure it out yourself"



AFTER: THE PROFESSIONAL PRODUCT

- Searchable and branded
- Rock-solid CI
- Welcoming community
- Prose documentation
- Clear contributor path

Don't Delay Writing Docs forever!



The Motivation: Why Bother?

YOU SHOULD, BECAUSE

- 1 **Self-Reflection:** Writing docs reveals API design flaws
- 2 **Better Design:** If it's hard to explain, the API is probably wrong
- 3 **Recognition:** Your hard work deserves users
- 4 **Wider Adoption:** Bleeding-edge code is only useful if people compile it
- 5 **Legacy:** Build something that outlives your interest

The Motivation: Why Bother?

YOU SHOULD, BECAUSE

- 1 **Self-Reflection:** Writing docs reveals API design flaws
- 2 **Better Design:** If it's hard to explain, the API is probably wrong
- 3 **Recognition:** Your hard work deserves users
- 4 **Wider Adoption:** Bleeding-edge code is only useful if people compile it
- 5 **Legacy:** Build something that outlives your interest

Better Docs = Better Code.

Call to Action

Don't try to implement all 10 tips at once.

Call to Action

Don't try to implement all 10 tips at once.

THIS WEEK

- Pick **ONE** tip that resonates
- Spend 2 hours implementing it
- Share your progress

Call to Action

Don't try to implement all 10 tips at once.

THIS WEEK

- Pick **ONE** tip that resonates
- Spend 2 hours implementing it
- Share your progress

NEXT MONTH

- Pick another tip
- Watch your community grow
- Iterate based on feedback

Call to Action

Don't try to implement all 10 tips at once.

THIS WEEK

- Pick **ONE** tip that resonates
- Spend 2 hours implementing it
- Share your progress

NEXT MONTH

- Pick another tip
- Watch your community grow
- Iterate based on feedback

Remember: Repo → Product is a journey, not a sprint.



The background is a solid yellow color. It is decorated with several black geometric shapes, primarily parallelograms and triangles, arranged in a pattern that suggests a 3D perspective or a stylized architectural design. These shapes are positioned around the edges and corners of the frame.

CAUTION
Programming
is addictive
(and too much fun)

